

CM3010 – Databases and Advanced Data Techniques

Midterm Assessment

World Cup Cricket Analytics Hub

A Comparative Analysis of ODI and T20 World Cup Formats

Student Information

Student Name: Dhanarasu Naveen

Student ID: 230655533

Course Code: CM3010

Assessment Type: Midterm Assessment

Submission Date: 7 January 2026

Project Resources

Live Application:

[Coursera Lab](#)

Data Source:

Cricsheet – ODI & T20 World Cup Match Data

<https://cricsheet.org/matches/>

Table Of Contents

Stage 1: Dataset Selection, Critique, and Research Motivation.....	4
1.1 Dataset Identification and Source.....	4
1.2 Project Scope and Subset Selection.....	4
1.3 Dataset Structure and Granularity.....	5
1.4 Research Motivation.....	6
1.5 Dataset Assessment and Critique.....	6
1.6 Research Questions.....	7
1.7 Summary of Stage 1 Outcomes.....	8
Stage 2: Data Modelling (E/R → Relational → Normalised Schema).....	9
2.1 Overview and Scope of the Model.....	9
2.2 E/R Diagrams (Conceptual + Relational Mapping).....	9
2.3 Conceptual E/R Model (High-Level Entities and Relationships).....	11
2.4 Logical Relational Model (Implementation-Ready Schema).....	12
2.5 Normalisation Evaluation (Up to at least 3NF).....	13
2.6 Summary.....	14
Stage 3 – Database Implementation and Analytical Query Design.....	15
3.1 Implementation of the Relational Database (MySQL).....	15
3.1.1 Implemented Schema Summary.....	15
3.1.2 Data Loading and Validation Approach.....	22
3.2 Mapping Research Questions to SQL Queries (Coverage Table).....	22
3.3 Analytical Query Implementation and Results.....	24
3.3.1 Questions Q1–Q6: Performance and Phase Analytics.....	24
3.3.2 Questions Q7–Q15: Context, Outcomes, Pressure, and Efficiency.....	35
3.4 Discussion of Query Design and Complexity.....	53
3.5 Link to Application-Level Usage (Preparation for Stage 4).....	54
3.6 Stage 3 Summary.....	54
Stage 4 – Web Application (Node.js Module).....	55
4.1 Purpose and Scope.....	55
4.2 Technology Stack.....	55
4.3 Application Architecture and Files.....	56
4.3.1 Backend Components.....	56
4.3.2 Frontend Components.....	56
4.4 Database Access and Privilege Considerations.....	57
4.5 User Flow and Application Behaviour.....	57
4.5.1 Query Hub (Home Page).....	57
4.5.2 Results Page (Per Question).....	58
4.6 Visual Analytics (Chart.js) Integration.....	58
4.7 Screenshots Included (Home + 6 Questions).....	58

4.8 Robustness and Error Handling.....	66
4.9 Evaluation Against Stage 4 Requirements.....	66
4.10 Summary.....	66

Stage 1: Dataset Selection, Critique, and Research Motivation

1.1 Dataset Identification and Source

This project uses ball-by-ball cricket match data from **Cricsheet**, an open-data provider that publishes structured match records for cricket in machine-readable formats. The dataset was obtained from the Cricsheet matches portal:

- <https://cricsheet.org/matches/>

Two match collections were downloaded in **JSON** format:

- **ODI (One Day International) matches** archive: 3073 matches
- **T20 (Twenty20 International) matches** archive: 4801 matches

Cricsheet notes that **some ODI matches are withheld** (152 matches) due to a policy change: matches involving the Afghanistan men's team or the Afghanistan Premier League are no longer distributed in the dataset. This is a source-level limitation (not introduced by the project) and is documented publicly by Cricsheet:

- <https://cricsheet.org/withheld-matches>

1.2 Project Scope and Subset Selection

Although the raw archives contain thousands of matches, the project aims to analyse **World Cup cricket only**, because World Cups provide:

- a controlled competitive context (high-stakes matches, consistent tournament structures),
- meaningful cross-team comparability,
- and a clear basis for comparative analysis between formats.

Therefore, the dataset was **externally filtered** (prior to database loading) to include only:

- **ODI World Cups:** 2011, 2015, 2019, 2023
- **T20 World Cups:** 2010, 2012, 2014, 2016, 2021, 2022, 2024

Final World Cup Subset Counts

ODI World Cups (Total = 166 matches)

- 2011: 49
- 2015: 42
- 2019: 36

- 2023: 39
Total ODI WC matches loaded: **166**

T20 World Cups (Total = 232 matches)

- 2010: 25
 - 2012: 25
 - 2014: 32
 - 2016: 27
 - 2021: 40
 - 2022: 39
 - 2024: 44
- Total T20 WC matches loaded: **232**

Combined dataset size (ODI + T20): 398 World Cup matches.

File Organisation in the Project

The curated dataset is stored in the project under a structured directory layout:

- `data/ODI_WC/{2011, 2015, 2019, 2023}/...`
- `data/T20_WC/{2010, 2012, 2014, 2016, 2021, 2022, 2024}/...`

This structure supports reproducibility: each tournament year is clearly separated and can be reloaded or validated independently.

1.3 Dataset Structure and Granularity

Cricsheet JSON match files are hierarchical and capture cricket at multiple levels, making them highly suitable for relational modelling:

- **Match-level metadata:** venue, city, date, event stage (group/knockout), toss, match outcome and margins
- **Team and player rosters:** participating teams and playing XI lists
- **Ball-by-ball events:** innings → overs → deliveries, including batter, bowler, non-striker, runs (bat + total), extras, wickets, and fielding involvement
- **Phase indicators:** powerplay ranges may be included for some matches (format-dependent)

The delivery-level detail enables deeper analytical questions than traditional match scorecards, including:

- phase-based performance (powerplay/middle/death),
- dismissal patterns by type and phase,
- bowling impact under pressure,

- fielding contributions (catches/run-outs),
- and match-context comparisons across ODI vs T20.

This granularity also motivates the use of a relational database: meaningful answers require joining match metadata with innings, deliveries, wickets, player entities, and team entities in a consistent schema. The filtered JSON files are subsequently ingested into a MySQL database using a custom Python ETL process, which normalises these entities into a structured relational schema. This ingestion step demonstrates that the dataset is not only theoretically suitable for relational modelling but also practically usable within a database-driven analytical application.

1.4 Research Motivation

Cricket is naturally suited to structured analytical study because the game is discrete and event-based: every ball produces measurable outcomes (runs, extras, wickets, phases, and roles). The project focuses on **World Cups** because they represent the highest-pressure tournament environment, where strategies, player roles, and team structures are tested against the best opposition.

The key motivation is comparative:

- **ODI World Cups** are longer-format games where batting depth, innings pacing, and sustained execution matter.
- **T20 World Cups** are shorter-format games where phase acceleration, death-over efficiency, and high-variance match events matter more.

By comparing ODI and T20 World Cups using the same modelling approach and consistent entity definitions (teams, players, deliveries, wickets, venues), the database can reveal structural differences between formats rather than isolated “best player” lists.

1.5 Dataset Assessment and Critique

Strengths

1. **High granularity:** Ball-by-ball structure enables phase-level and role-level analysis (batting, bowling, fielding) rather than relying on aggregate scorecards
2. **Real-world data:** Matches represent authentic tournament outcomes and competitive contexts.
3. **Rich match context:** Includes tournament stage, outcome margins, venue metadata, and playing XIs, supporting multi-dimensional research.
4. **Relational suitability:** The hierarchy naturally decomposes into entities (Match, Innings, Delivery, Player, Team, Wicket, Extras, FieldingEvent), enabling complex SQL queries.

Limitations and Data Quality Risks

1. **Source-level incompleteness (withheld matches):** Cricsheet excludes certain ODI matches (152 withheld) due to policy restrictions involving Afghanistan men and the Afghanistan Premier League. This may reduce completeness for analyses involving Afghanistan participation, and should be treated as an external availability constraint rather than an error in project processing.
2. **Optional fields and schema variability:** Not all deliveries contain extras or wickets; fielders may be absent; and some metadata fields (e.g., stage) may be missing or inconsistently populated depending on match documentation.
3. **Entity ambiguity (player identity):** Player names are strings in the raw data and can introduce duplication risks (e.g., spelling variants, same-name collisions). Cricsheet registry identifiers may exist, but many practical database builds still require careful normalisation decisions.
4. **Venue naming consistency:** Venues and cities can be inconsistent across tournaments (e.g., formatting differences), requiring normalisation in the database layer.
5. **Comparability across formats:** ODI and T20 differ in match length, phase definitions, and tactical pacing. Comparisons must use format-aware rules (e.g., powerplay boundaries), which the database queries must explicitly encode.

Mitigation Strategy (high-level)

To reduce these risks, the project applies:

- controlled scope to World Cups only (clear boundaries),
- systematic normalisation into core entities (Team, Player, Venue, Match),
- delivery-level ingestion with separate tables for extras, wickets, and fielding events,
- and consistent phase logic in queries for ODI vs T20 comparisons.

1.6 Research Questions

The database is designed to answer research questions that require multi-table relational joins across matches, innings, deliveries, players, teams, wickets, extras, phases, venues, and match outcomes. The project investigates performance patterns across ODI and T20 World Cups through the following questions:

1. Which players exhibit the most consistent run-scoring patterns across World Cup matches, when accounting for both scoring variability and career longevity, and how does this consistency differ between ODI and T20 formats?
2. How does batting productivity, measured by strike rate, vary across powerplay, middle, and death overs for top run-scoring batters in ODI and T20 World Cups?
3. How does batting strike rate vary across different innings phases for top-order and middle-order batters in ODI and T20 World Cups?

4. Which teams gain the largest proportion of their total runs from powerplay overs in ODI versus T20 World Cups, and how does this differ between winning and losing matches?
5. How does wicket-taking effectiveness vary between powerplay and death overs across formats, and which bowlers specialise in each phase?
6. What dismissal types dominate in ODI and T20 World Cups, and how does their distribution vary across match phases?
7. Which players deliver the highest combined batting and bowling impact in World Cup knockout matches?
8. How do fielding contributions (catches and run-outs) differ between ODI and T20 World Cup knockout matches, and which teams benefit most from these events?
9. How does team scoring progression across match phases differ between ODI and T20 World Cups, and what structural differences emerge between the formats?
10. How do average first- and second-innings scores differ by venue in ODI and T20 World Cups, and how consistent are these scoring patterns across tournaments?
11. How does bowling economy in death overs compare between ODI and T20 World Cups, and which bowlers consistently limit scoring under end-of-innings pressure?
12. How does top-order contribution to total team runs differ between ODI and T20 World Cups, and is early batting dominance more strongly associated with winning in one format than the other?
13. How does the contribution of extras to total team runs vary between ODI and T20 World Cups, and how does the distribution of different extra types differ across formats?
14. How does run-chase efficiency, measured as the ratio of achieved to required run rate in successful chases, differ between teams in ODI and T20 World Cups?
15. How does the timing of wicket losses affect scoring momentum in ODI and T20 World Cups, as measured by changes in run rate immediately before and after wickets?

1.7 Summary of Stage 1 Outcomes

Stage 1 establishes a clear dataset foundation and justifies the database approach:

- A large raw archive was narrowed to a well-defined, high-value analytical subset (398 World Cup matches).
 - The JSON structure supports deep relational modelling at match, innings, and delivery level.
 - Limitations (including withheld matches and optional fields) are acknowledged and mitigated through modelling choices.
 - Research questions are explicitly designed to leverage relational joins and phase-aware cricket logic, enabling meaningful comparative analysis of ODI vs T20 World Cups.
-

Stage 2: Data Modelling (E/R → Relational → Normalised Schema)

2.1 Overview and Scope of the Model

This project models open, real-world ball-by-ball cricket match data sourced from Cricsheet. A single match contains match metadata (format, venue, date, stage, participating teams, winner/result), team/player participation, and delivery-level events including striker/bowler identities, runs off the bat, extras, and dismissals.

The dataset is inherently hierarchical (Match → Innings → Delivery → events such as extras/wickets/fielding). It also contains repeated real-world entities (teams, players, venues) appearing across many matches. A relational design is therefore appropriate because it:

- de-duplicates repeated entities (e.g., a player appearing across many matches),
- enforces integrity between hierarchy levels (e.g., a Delivery must belong to an Innings, and an Innings must belong to a Match),
- supports flexible analytical queries across formats and tournaments.

The implemented schema follows a normalised structure where static reference data (Format, Team, Venue) is separated from high-volume event data (Delivery, Wicket, DeliveryExtra). This reduces redundancy, improves consistency, and supports accurate aggregation.

2.2 E/R Diagrams (Conceptual + Relational Mapping)

Two E/R diagrams are provided to match the module's modelling expectations and to demonstrate E/R-to-relational mapping clearly.

Diagram 1: Conceptual E/R (real-world model using rectangles/diamonds/ellipses)

This diagram represents the domain relationships as they occur naturally. In particular, the relationship between **Team** and **Match** is conceptually **many-to-many (M:N)**: a team plays many matches, and each match contains multiple teams (two in standard World Cup matches). Conceptual E/R notation is the correct place to represent this M:N relationship directly.

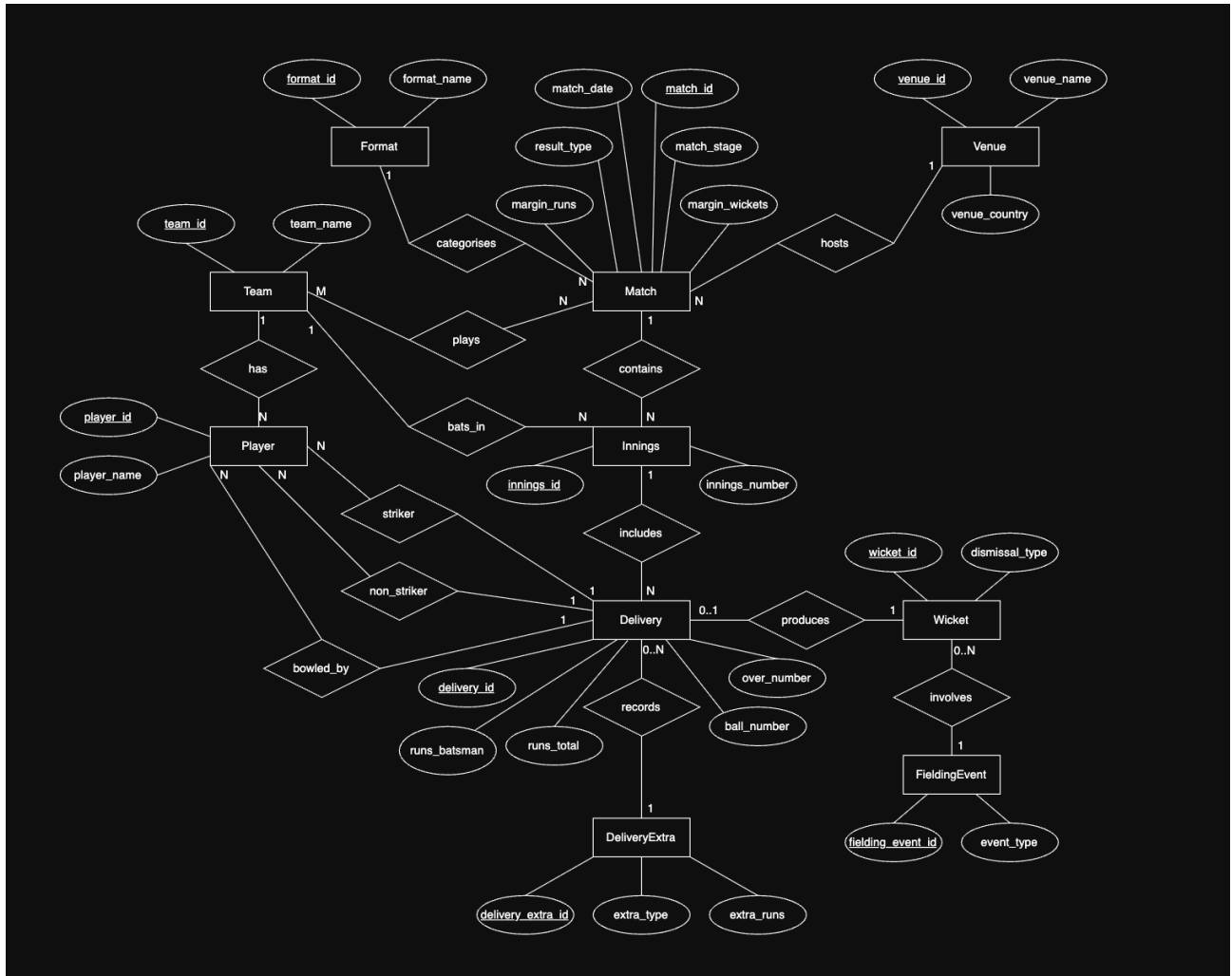
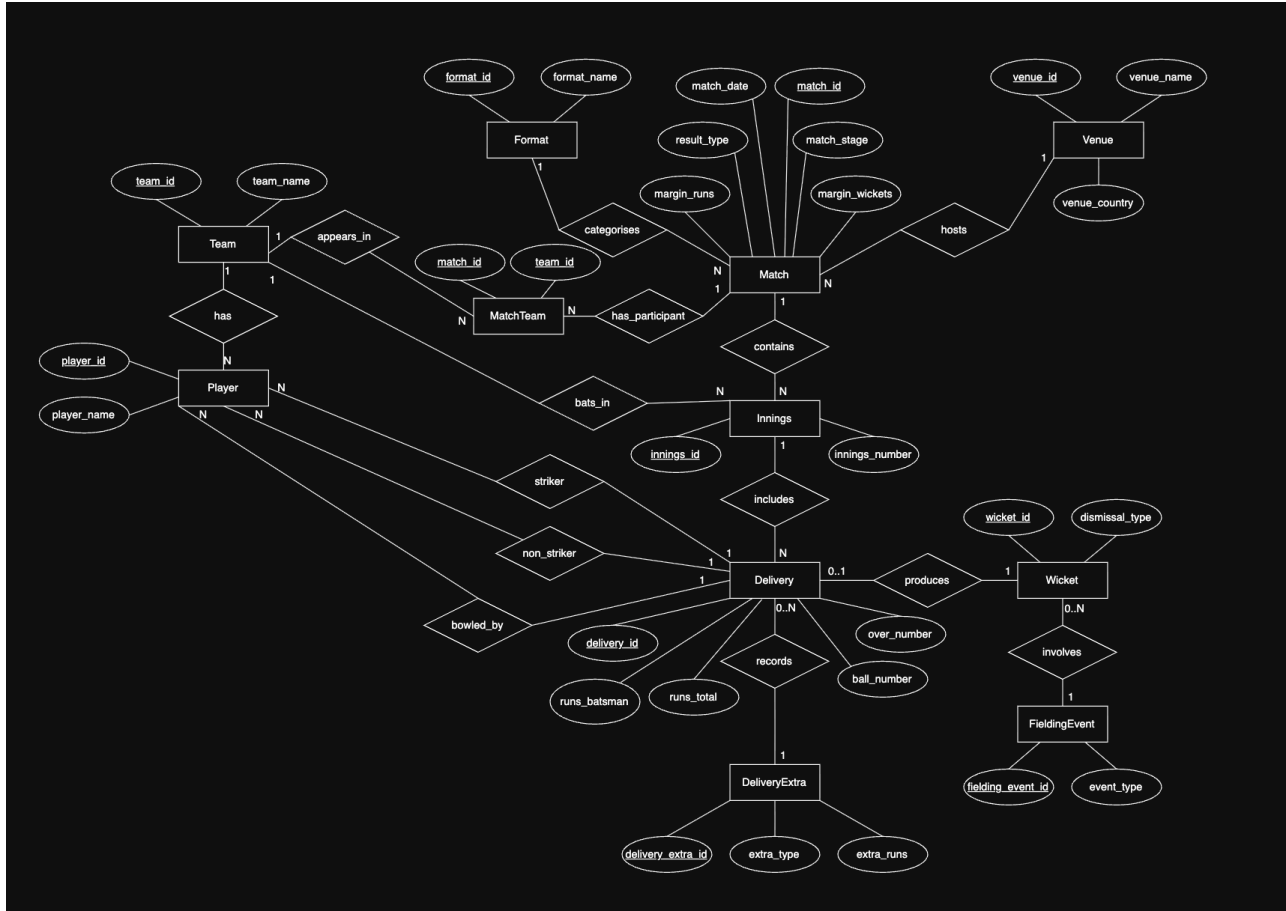


Diagram 2: Logical Relational Model (implementation-ready structure with PK/FK)

Relational databases cannot store M:N relationships directly without introducing an associative entity. Therefore, the M:N Team–Match relationship is resolved using the junction table **MatchTeam(match_id, team_id)**. This second diagram shows the modified, relationally compatible structure and documents the mapping from conceptual design to SQL implementation.



Providing both diagrams reduces ambiguity for a strict marker and explicitly demonstrates the required transformation from E/R relationships into a relational schema.

2.3 Conceptual E/R Model (High-Level Entities and Relationships)

The conceptual model includes the following primary entities and relationships:

Entities

- **Format:** Identifies match format (e.g., ODI, T20).
- **Venue:** Stores stadium and host country information.
- **Team:** Represents national teams participating in World Cups.
- **Player:** Represents players, each belonging to one Team.
- **Match:** Represents a single match, linked to one Format and one Venue, and optionally a winner Team.
- **Innings:** Represents innings within a match, associated with a batting Team.
- **Delivery:** Represents the most granular event: a ball bowled within an innings, including striker/non-striker/bowler and runs.
- **Wicket:** Represents a dismissal event that occurs on a specific delivery.

- **DeliveryExtra:** Represents extras recorded on a specific delivery.
- **FieldingEvent:** Represents fielding involvement tied to a wicket (e.g., catch, stumping, run out involvement).

Key Relationships (Conceptual)

- **Team–Player:** One Team has many Players; each Player belongs to one Team (1:N).
- **Format–Match:** One Format is used by many Matches; each Match has one Format (1:N).
- **Venue–Match:** One Venue hosts many Matches; each Match is at one Venue (1:N).
- **Team–Match:** A Team plays many Matches and each Match includes multiple Teams (M:N).
- **Match–Innings:** One Match has many Innings; each Innings belongs to one Match (1:N).
- **Team–Innings:** One Team can bat in many Innings; each Innings has one batting Team (1:N).
- **Innings–Delivery:** One Innings contains many Deliveries; each Delivery belongs to one Innings (1:N).
- **Delivery–Wicket:** A Delivery may produce zero or one Wicket; each Wicket must belong to one Delivery (0..1 : 1).
- **Delivery–DeliveryExtra:** A Delivery may have zero or more Extras; each DeliveryExtra must belong to one Delivery (0..N : 1).
- **Wicket–FieldingEvent:** A Wicket may have zero or more fielding involvements; each FieldingEvent belongs to one Wicket (0..N : 1).

This conceptual structure captures the real-world domain without forcing relational implementation constraints, which are handled in the logical model.

2.4 Logical Relational Model (Implementation-Ready Schema)

The final implemented schema resolves conceptual relationships into normalised relational tables with primary and foreign keys:

Lookup / Reference Tables

- **Format(format_id PK, format_name UNIQUE)**
- **Venue(venue_id PK, venue_name, venue_country)**
- **Team(team_id PK, team_name UNIQUE)**

Core Tables

- **Player**(player_id PK, player_name, team_id FK → Team.team_id)
- **Match**(match_id PK, format_id FK → Format.format_id, venue_id FK → Venue.venue_id, match_stage, match_date, winner_team_id FK → Team.team_id, result_type, margin_runs, margin_wickets)

Associative / Structural Tables

- **MatchTeam**(match_id FK → Match.match_id, team_id FK → Team.team_id, PK(match_id, team_id))
Resolves the conceptual M:N relationship between Team and Match.
- **Innings**(innings_id PK, match_id FK → Match.match_id, batting_team_id FK → Team.team_id, innings_number, UNIQUE(match_id, innings_number))

Transactional / Event Tables

- **Delivery**(delivery_id PK, innings_id FK → Innings.innings_id, over_number, ball_number, bowler_id FK → Player.player_id, striker_id FK → Player.player_id, non_striker_id FK → Player.player_id, runs_batsman, runs_total, UNIQUE(innings_id, over_number, ball_number))
- **DeliveryExtra**(delivery_extra_id PK, delivery_id FK → Delivery.delivery_id, extra_type, extra_runs)
- **Wicket**(wicket_id PK, delivery_id FK → Delivery.delivery_id, dismissed_player_id FK → Player.player_id, dismissal_type)
- **FieldingEvent**(fielding_event_id PK, wicket_id FK → Wicket.wicket_id, fielder_id FK → Player.player_id, event_type)

This logical design ensures referential integrity across the hierarchy (Match → Innings → Delivery) and supports analytical queries that require joining match context with ball-by-ball events.

2.5 Normalisation Evaluation (Up to at least 3NF)

The schema is designed to satisfy at least **Third Normal Form (3NF)**.

First Normal Form (1NF)

All tables store atomic values (no repeating groups, no lists in a single attribute). Repeating delivery-attributes such as extras are stored in a separate table (**DeliveryExtra**) rather than embedded as multiple columns or multi-valued fields.

Second Normal Form (2NF)

All non-key attributes are fully dependent on the whole primary key. For composite keys, the only table using a composite primary key is **MatchTeam(match_id, team_id)**, and it contains no non-key attributes, so partial dependencies do not exist.

Third Normal Form (3NF)

Non-key attributes do not depend transitively on other non-key attributes:

- Match attributes (date, stage, result details) depend only on **match_id**, while Format and Venue details are separated into their own tables and referenced via foreign keys.
- Player details are separated from Team details; team names are stored only in **Team** and referenced through **Player.team_id**.
- Delivery-level events are separated into distinct event tables (**DeliveryExtra**, **Wicket**, **FieldingEvent**) to avoid mixed dependencies and duplication.

BCNF / Higher Normal Forms (Justification)

No functional dependencies are identified where a non-candidate determinant controls other attributes inside a table (beyond primary/candidate keys), and the design avoids multi-valued dependencies by modelling extras and fielding involvement as separate entities. As a result, further normalisation beyond 3NF is not required for this analytical workload, and the current schema provides a strong balance of integrity, clarity, and query performance.

2.6 Summary

Stage 2 produced:

1. a **Conceptual E/R diagram** capturing real-world relationships (including M:N Team–Match), and
2. a **Relational/Logical model** resolving relational incompatibilities using an associative entity (**MatchTeam**) and implementing a normalised schema suitable for MySQL.

The final design supports both high-level match analysis and detailed ball-by-ball event analytics while maintaining strong data integrity through keys, constraints, and normalised structure.

Stage 3 – Database Implementation and Analytical Query Design

3.1 Implementation of the Relational Database (MySQL)

Stage 2 defined a normalised relational schema for World Cup cricket analytics. In Stage 3, this schema was implemented in **MySQL** and used to store ball-by-ball match data in a way that supports both statistical analysis and application-level querying.

For readability and to remain within report length constraints, SQL queries are presented in compacted form. The full, executable versions of all queries are included in the project repository and were used to generate the reported results.

3.1.1 Implemented Schema Summary

The database is structured as a **hierarchical match** → **innings** → **delivery** model, supported by reference and event tables.

Lookup / reference tables

- **Format(format_id, format_name)**: separates ODI vs T20 logic cleanly without hard-coding assumptions.
- **Venue(venue_id, venue_name, venue_country)**: standardises location metadata for venue bias analysis.
- **Team(team_id, team_name)**: canonical team identities for joins across all match records.

Core entities

- **Player(player_id, player_name, team_id)**: links each player to a team (FK to Team).
- **Match(match_id, format_id, venue_id, match_stage, match_date, winner_team_id, ...)**: stores match metadata and outcomes (FK to Format, Venue, Team).

Relational / structural tables

- **MatchTeam(match_id, team_id)**: resolves the many-to-many relationship between Match and Team (each match has two teams, each team appears in many matches).
- **Innings(innings_id, match_id, batting_team_id, innings_number)**: creates a structured innings layer (FK to Match and Team), ensuring uniqueness via **UNIQUE(match_id, innings_number)**.

Transactional / event tables

- **Delivery**(**delivery_id**, **innings_id**, **over_number**, **ball_number**, **bowler_id**, **striker_id**, **non_striker_id**, **runs_batsman**, **runs_total**)
The granular fact table, constrained with **UNIQUE**(**innings_id**, **over_number**, **ball_number**) to prevent duplicated balls.
- **DeliveryExtra**(**delivery_extra_id**, **delivery_id**, **extra_type**, **extra_runs**): stores extras at delivery-level without bloating Delivery.
- **Wicket**(**wicket_id**, **delivery_id**, **dismissed_player_id**, **dismissal_type**): records wicket events tied to deliveries.
- **FieldingEvent**(**fielding_event_id**, **wicket_id**, **fielder_id**, **event_type**): captures fielder involvement (catches, run-outs, stumpings) where applicable.

This implementation ensures:

- **Data integrity** through primary keys, foreign keys, and uniqueness constraints.
- **Analytical flexibility** by separating transactional events (Delivery, Wicket, Extras) from identity metadata (Team, Player, Venue, Format).
- **Compatibility with advanced SQL** (CTEs and window functions) needed for deeper cricket analytics.

Implemented SQL Schema (DDL)

The following SQL Data Definition Language (DDL) was used to instantiate the relational schema described above in MySQL.

```
/* * DATABASE SCHEMA: Cricket World Cup Analytics

* -----

* Description: Relational structure for tracking multi-format World Cup data.

* Architecture: normalized Snowflake Schema design to ensure data integrity

* from high-level Match details down to ball-by-ball Delivery level.

* -----

*/

CREATE DATABASE IF NOT EXISTS cricket_wc;

USE cricket_wc;
```



```
-- =====  
-- 1. LOOKUP TABLES (Static Metadata)  
-- =====  
  
-- Stores match formats (e.g., 'ODI', 'T20')  
  
CREATE TABLE Format (  
    format_id INT AUTO_INCREMENT PRIMARY KEY,  
    format_name VARCHAR(20) NOT NULL UNIQUE  
);  
  
-- Stores stadium information and locations  
  
CREATE TABLE Venue (  
    venue_id INT AUTO_INCREMENT PRIMARY KEY,  
    venue_name VARCHAR(100) NOT NULL,  
    venue_country VARCHAR(100) NOT NULL  
);  
  
-- Stores participating national teams  
  
CREATE TABLE Team (  
    team_id INT AUTO_INCREMENT PRIMARY KEY,  
    team_name VARCHAR(100) NOT NULL UNIQUE  
);  
  
-- =====
```

```
-- 2. CORE ENTITIES

-- =====

-- Individual player profiles linked to their respective teams

CREATE TABLE Player (

    player_id INT AUTO_INCREMENT PRIMARY KEY,

    player_name VARCHAR(100) NOT NULL,

    team_id INT NOT NULL,

    FOREIGN KEY (team_id) REFERENCES Team(team_id)

);

-- Primary Match record tracking stages, dates, and outcomes

CREATE TABLE `Match` (

    match_id INT AUTO_INCREMENT PRIMARY KEY,

    format_id INT NOT NULL,

    venue_id INT NOT NULL,

    match_stage VARCHAR(50) NOT NULL, -- e.g., Group, Semi Final, Final

    match_date DATE NOT NULL,

    winner_team_id INT,

    result_type VARCHAR(50), -- e.g., normal, tie, no result

    margin_runs INT,

    margin_wickets INT,

    FOREIGN KEY (format_id) REFERENCES Format(format_id),

    FOREIGN KEY (venue_id) REFERENCES Venue(venue_id),
```

```

FOREIGN KEY (winner_team_id) REFERENCES Team(team_id)

);

-- =====

-- 3. ASSOCIATIVE & STRUCTURAL TABLES

-- =====

-- Many-to-Many mapping to link two teams to a single match

CREATE TABLE MatchTeam (

    match_id INT NOT NULL,

    team_id INT NOT NULL,

    PRIMARY KEY (match_id, team_id),

    FOREIGN KEY (match_id) REFERENCES `Match`(match_id),

    FOREIGN KEY (team_id) REFERENCES Team(team_id)

);

-- Bridges Matches and Deliveries; tracks batting order (1st/2nd Innings)

CREATE TABLE Innings (

    innings_id INT AUTO_INCREMENT PRIMARY KEY,

    match_id INT NOT NULL,

    batting_team_id INT NOT NULL,

    innings_number INT NOT NULL,

    FOREIGN KEY (match_id) REFERENCES `Match`(match_id),

    FOREIGN KEY (batting_team_id) REFERENCES Team(team_id),

```

```

    UNIQUE (match_id, innings_number)

);

-- =====

-- 4. TRANSACTIONAL DATA (Ball-by-Ball)

-- =====

-- The most granular table: tracks every legal and illegal ball bowled

CREATE TABLE Delivery (

    delivery_id INT AUTO_INCREMENT PRIMARY KEY,

    innings_id INT NOT NULL,

    over_number INT NOT NULL,

    ball_number INT NOT NULL,

    bowler_id INT NOT NULL,

    striker_id INT NOT NULL,

    non_striker_id INT NOT NULL,

    runs_batsman INT NOT NULL DEFAULT 0, -- Runs off the bat

    runs_total INT NOT NULL DEFAULT 0,   -- Total runs including extras

    FOREIGN KEY (innings_id) REFERENCES Innings(innings_id),

    FOREIGN KEY (bowler_id) REFERENCES Player(player_id),

    FOREIGN KEY (striker_id) REFERENCES Player(player_id),

    FOREIGN KEY (non_striker_id) REFERENCES Player(player_id),

    UNIQUE (innings_id, over_number, ball_number)

);

```

```
-- Sub-table for Deliveries: tracks types of extras (Wides, No Balls, etc.)
```

```
CREATE TABLE DeliveryExtra (  
  
    delivery_extra_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    delivery_id INT NOT NULL,  
  
    extra_type VARCHAR(20) NOT NULL,  
  
    extra_runs INT NOT NULL,  
  
    FOREIGN KEY (delivery_id) REFERENCES Delivery(delivery_id)  
  
);
```

```
-- Records wicket events linked to a specific delivery
```

```
CREATE TABLE Wicket (  
  
    wicket_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    delivery_id INT NOT NULL,  
  
    dismissed_player_id INT NOT NULL,  
  
    dismissal_type VARCHAR(50) NOT NULL,  
  
    FOREIGN KEY (delivery_id) REFERENCES Delivery(delivery_id),  
  
    FOREIGN KEY (dismissed_player_id) REFERENCES Player(player_id)  
  
);
```

```
-- Records fielder involvement in a wicket (Catches, Run Outs, Stumpings)
```

```
CREATE TABLE FieldingEvent (  
  
    fielding_event_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    wicket_id INT NOT NULL,
```

```

    fielder_id INT NOT NULL,

    event_type VARCHAR(30) NOT NULL,

    FOREIGN KEY (wicket_id) REFERENCES Wicket(wicket_id),

    FOREIGN KEY (fielder_id) REFERENCES Player(player_id)

);

```

3.1.2 Data Loading and Validation Approach

After creating the schema, match data was loaded into MySQL using a **custom Python ETL script (load_data.py)** that parses Cricsheet JSON files and inserts records into the database using **parameterised INSERT statements**. Data were loaded in a controlled sequence to satisfy foreign key dependencies:

1. Lookup tables (Format, Venue, Team)
2. Player
3. Match and MatchTeam
4. Innings
5. Delivery
6. DeliveryExtra, Wicket, FieldingEvent

Basic validation checks were performed to confirm correct loading and relational consistency, such as:

- verifying each Match has exactly two records in MatchTeam,
- confirming Innings uniqueness per match (1st/2nd innings),
- confirming Delivery uniqueness by innings/over/ball,
- ensuring that wicket and extra records always reference valid deliveries.

3.2 Mapping Research Questions to SQL Queries (Coverage Table)

All **15 research questions** identified in Stage 1 were implemented as SQL queries (stored in the project SQL repository as **queries.sql**). Table 3.1 shows the full mapping between questions and the schema components used.

Table 3.1 – Research Question → Query Coverage and Core Tables

Question	Analytical Focus	Main Metric / Output	Core Tables Used
Q1	High-impact batters	Weighted impact score (consistency × longevity)	Delivery, Innings, Match, Player, Format
Q2	Phase batting productivity	Runs + strike rate by phase (PP/Middle/Death)	Delivery, Innings, Match, Player, Team, Format
Q3	Role-based pacing	Strike rate by phase for Top vs Middle order	Delivery, Innings, Match, Player, Team, Format
Q4	Powerplay influence on outcome	Powerplay share of total runs (Win vs Loss)	Delivery, DeliveryExtra, Innings, Match, Team, Format
Q5	Bowling effectiveness by phase	Balls-per-wicket (PP vs Death), excluding run-outs	Delivery, Innings, Match, Player, Wicket, MatchTeam, Team, Format
Q6	Dismissal distribution	Wicket types by phase with percentage share	Wicket, Delivery, Innings, Match, Format
Q7	Knockout impact players	Combined batting + bowling normalised impact	Match, Innings, Delivery, Wicket, Player, MatchTeam, Team
Q8	Fielding contributions	Catches/run-outs per team in knockouts	Match, Innings, Delivery, Wicket, MatchTeam, Team, Format
Q9	Team scoring progression	Run rate by phase for top teams	Delivery, Innings, Match, Team, Format
Q10	Venue scoring bias	Avg 1st vs 2nd innings runs by venue	Delivery, Innings, Match, Venue, Format
Q11	Death-over bowling economy	Economy rate in final overs (ODI/T20)	Delivery, Innings, Match, Player, Wicket, MatchTeam, Team, Format
Q12	Top-order stability vs win rate	Win% by runs-before-2nd-wicket bands	Delivery, Wicket, Innings, Match, Team, Format

Q13	Extras contribution	Extras by type + % share of team runs	DeliveryExtra, Delivery, Innings, Match, Team, Format
Q14	Chase efficiency	Chase success% + RR efficiency ratio	Match, Innings, Delivery, DeliveryExtra, Team, Format
Q15	Momentum impact after wickets	RR change pre/post wicket windows	Wicket, Delivery, Innings, Match, Team, Format

This table demonstrates complete alignment between Stage 1 research objectives and the implemented SQL analytics suite.

3.3 Analytical Query Implementation and Results

This section documents the SQL implementation for each of the 15 questions. For consistency and clarity, each query is presented using the same structure:

- research aim and rationale,
- logic summary (joins, filters, phases, window functions),
- expected output fields,
- placeholders for the SQL listing and the result screenshot/table (added after execution).

Note: The full SQL text for every question is stored in *queries.sql*. Each listing and result image will be inserted directly under its corresponding question.

3.3.1 Questions Q1–Q6: Performance and Phase Analytics

Q1 – High-Impact Batters (Longevity vs Consistency)

Research aim: Identify batters who combine repeat high-impact innings with sustained World Cup participation across formats.

Logic summary: Aggregate player runs per match; label “high-impact innings” using format-specific thresholds (ODI 50+, T20 30+); apply a logarithmic weighting based on matches played to reward longevity; rank top performers per format.

Key tables: Delivery, Innings, Match, Player, Format.

Output fields: format_name, player_name, matches_played, high_impact_innings, impact_consistency_rate, weighted_impact_score, avg_runs_per_match.

SQL Query:

```
WITH player_match_runs AS (
    SELECT f.format_name, m.match_id, p.player_id, p.player_name, SUM(d.runs_batsman)
    AS runs_in_match
```



```

    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id
    JOIN `Match` m ON i.match_id = m.match_id JOIN Format f ON m.format_id =
f.format_id
    JOIN Player p ON d.striker_id = p.player_id
    GROUP BY f.format_name, m.match_id, p.player_id, p.player_name
),
player_total_runs AS (
    SELECT format_name, player_id, player_name, SUM(runs_in_match) AS total_runs
    FROM player_match_runs GROUP BY format_name, player_id, player_name
),
top_run_scorers AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY format_name ORDER BY total_runs DESC) AS
run_rank
    FROM player_total_runs
),
player_impact AS (
    SELECT pmr.format_name, pmr.player_id, pmr.player_name, COUNT(pmr.match_id) AS
matches_played,
    SUM(CASE WHEN (pmr.format_name = 'ODI' AND pmr.runs_in_match >= 50) OR
(pmr.format_name = 'T20' AND pmr.runs_in_match >= 30) THEN 1 ELSE 0 END) AS
high_impact_innings,
    AVG(pmr.runs_in_match) AS avg_runs_per_match
    FROM player_match_runs pmr JOIN top_run_scorers trs ON pmr.player_id =
trs.player_id AND pmr.format_name = trs.format_name
    WHERE trs.run_rank <= 15 GROUP BY pmr.format_name, pmr.player_id, pmr.player_name
),
ranked_players AS (
    SELECT format_name, player_name, matches_played, high_impact_innings,
    ROUND(high_impact_innings / matches_played, 3) AS impact_consistency_rate,
    ROUND(high_impact_innings * LOG(matches_played), 3) AS weighted_impact_score,
    ROUND(avg_runs_per_match, 2) AS avg_runs_per_match,
    ROW_NUMBER() OVER (PARTITION BY format_name ORDER BY (high_impact_innings *
LOG(matches_played)) DESC, avg_runs_per_match DESC) AS impact_rank
    FROM player_impact WHERE matches_played >= 5
)
SELECT format_name, player_name, matches_played, high_impact_innings,
impact_consistency_rate, weighted_impact_score, avg_runs_per_match
FROM ranked_players WHERE impact_rank <= 5 ORDER BY format_name, impact_rank;

```

Output:

format_name	player_name	matches_played	high_impact_innings	impact_consistency_rate	weighted_impact_score	avg_runs_per_match
ODI	V Kohli	35	15	0.429	53.33	47.88
ODI	RG Sharma	26	12	0.462	39.097	59.58
ODI	SPD Smith	28	10	0.357	33.322	36.54
ODI	DA Warner	26	9	0.346	29.323	47.77
ODI	Shakib Al Hasan	24	9	0.375	28.682	41.75
T20	V Kohli	31	17	0.548	58.378	39.29
T20	JC Buttler	31	12	0.387	41.208	31.42
T20	RG Sharma	33	11	0.333	38.462	27.82
T20	Shakib Al Hasan	33	11	0.333	38.462	23.86
T20	DA Warner	37	18	0.278	36.189	24.14

18 rows in set (8.59 sec)

Short analysis:

Consistent run-scoring in World Cups is driven by both longevity and repeatable high-impact innings. ODI formats reward sustained accumulation across matches, while T20 formats show higher variance and fewer consistently dominant batters. Players who rank highly in both formats demonstrate adaptability to different pacing and risk structures.

Q2 – Phase-Based Batting Productivity (Efficiency Mapping)

Research aim: Compare elite batters' phase productivity and strike rates across Powerplay, Middle, and Death overs for ODI and T20.

Logic summary: Identify top run scorers per format; map deliveries to phases using format-specific over windows; compute runs, balls faced, strike rate; apply minimum sample-size filters for stability; rank top performers per phase.

Key tables: Delivery, Innings, Match, Player, Team, Format.

Output fields: format_name, match_phase, player_name, team_name, matches_played, total_runs, balls_faced, strike_rate.

SQL Query:

```
WITH player_total_runs AS (
    SELECT f.format_name, p.player_id, p.player_name, SUM(d.runs_batsman) AS total_runs
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id
    JOIN `Match` m ON i.match_id = m.match_id JOIN Format f ON m.format_id =
f.format_id
    JOIN Player p ON d.striker_id = p.player_id GROUP BY 1, 2, 3
),
elite_batters AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY format_name ORDER BY total_runs DESC) AS
run_rank FROM player_total_runs
),
phase_deliveries AS (
    SELECT f.format_name, p.player_name, t.team_name, m.match_id, d.runs_batsman, 1 AS
ball_faced,
    CASE WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 1 AND 10 THEN 'Powerplay'
```

```

        WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 11 AND 40 THEN 'Middle
Overs'

        WHEN f.format_name = 'ODI' AND d.over_number >= 41 THEN 'Death Overs'

        WHEN f.format_name = 'T20' AND d.over_number BETWEEN 1 AND 6 THEN 'Powerplay'

        WHEN f.format_name = 'T20' AND d.over_number BETWEEN 7 AND 15 THEN 'Middle
Overs'

        WHEN f.format_name = 'T20' AND d.over_number >= 16 THEN 'Death Overs' END AS
match_phase

    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id
    JOIN `Match` m ON i.match_id = m.match_id JOIN Format f ON m.format_id =
f.format_id
    JOIN Player p ON d.striker_id = p.player_id JOIN Team t ON p.team_id = t.team_id
    JOIN elite_batters eb ON eb.player_id = p.player_id AND eb.format_name =
f.format_name WHERE eb.run_rank <= 15
),
phase_aggregates AS (
    SELECT format_name, match_phase, player_name, team_name, COUNT(DISTINCT match_id)
AS matches_played,
    SUM(runs_batsman) AS total_runs, SUM(ball_faced) AS balls_faced,
ROUND((SUM(runs_batsman) / SUM(ball_faced)) * 100, 2) AS strike_rate
    FROM phase_deliveries WHERE match_phase IS NOT NULL GROUP BY 1, 2, 3, 4
),
ranked_phase_players AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY format_name, match_phase ORDER BY
total_runs DESC) AS phase_rank
    FROM phase_aggregates WHERE (format_name = 'ODI' AND ((match_phase = 'Powerplay'
AND balls_faced >= 30) OR (match_phase = 'Middle Overs' AND balls_faced >= 70) OR
(match_phase = 'Death Overs' AND balls_faced >= 30)))
    OR (format_name = 'T20' AND ((match_phase = 'Powerplay' AND balls_faced >= 18) OR
(match_phase = 'Middle Overs' AND balls_faced >= 30) OR (match_phase = 'Death Overs'
AND balls_faced >= 12)))
)
SELECT format_name, match_phase, player_name, team_name, matches_played, total_runs,
balls_faced, strike_rate
FROM ranked_phase_players WHERE phase_rank <= 5 ORDER BY 1, 2, phase_rank;

```

Output:

format_name	match_phase	player_name	team_name	matches_played	total_runs	balls_faced	strike_rate
ODI	Death Overs	Mahmudullah	Bangladesh	11	266	225	118.22
ODI	Death Overs	V Kohli	India	11	218	159	132.88
ODI	Death Overs	AB de Villiers	South Africa	6	165	61	270.49
ODI	Death Overs	KS Williamson	New Zealand	5	133	183	129.13
ODI	Death Overs	SPD Smith	Australia	9	188	96	112.58
ODI	Middle Overs	V Kohli	India	30	1285	1429	84.32
ODI	Middle Overs	Shakib Al Hasan	Bangladesh	21	854	974	87.68
ODI	Middle Overs	KC Sangakkara	Sri Lanka	13	882	855	93.88
ODI	Middle Overs	KS Williamson	New Zealand	22	734	878	84.37
ODI	Middle Overs	JE Root	England	18	724	826	87.65
ODI	Powerplay	RG Sharma	India	24	584	576	101.39
ODI	Powerplay	DA Warner	Australia	25	587	542	93.54
ODI	Powerplay	HJ Guptill	New Zealand	24	412	538	76.58
ODI	Powerplay	Q de Kock	South Africa	23	359	444	80.86
ODI	Powerplay	TM Dilshan	Sri Lanka	15	345	375	92.08
T20	Death Overs	V Kohli	India	13	237	125	189.68
T20	Death Overs	EJG Morgan	England	14	162	99	163.64
T20	Death Overs	JC Buttler	England	8	134	68	197.06
T20	Death Overs	Shakib Al Hasan	Bangladesh	7	79	53	149.06
T20	Death Overs	MN Samuels	West Indies	6	74	48	154.17
T20	Middle Overs	V Kohli	India	22	685	486	124.49
T20	Middle Overs	Shakib Al Hasan	Bangladesh	26	466	388	122.63
T20	Middle Overs	JC Buttler	England	21	427	298	147.24
T20	Middle Overs	KS Williamson	New Zealand	17	358	279	125.45
T20	Middle Overs	EJG Morgan	England	18	315	271	116.24
T20	Powerplay	RG Sharma	India	29	498	373	131.37
T20	Powerplay	DA Warner	Australia	32	485	359	135.18
T20	Powerplay	JC Buttler	England	19	365	262	139.31
T20	Powerplay	Q de Kock	South Africa	21	359	248	144.76
T20	Powerplay	V Kohli	India	28	352	338	106.67

38 rows in set (0.58 sec)

Short analysis:

ODI batting productivity is concentrated in the middle overs, where strike-rate stability enables high run volume with controlled risk. In contrast, T20 batting shows elevated strike rates across all phases, with death overs producing the most aggressive scoring. These patterns reflect format-specific incentives rather than differences in batting quality.

Q3 – Strike Rate by Batting Order and Phase (Role-Based Pacing)

Research aim: Evaluate how scoring pace differs between top-order (1–3) and middle-order (4–6) batters across match phases.

Logic summary: Determine batting entry using first-ball faced per innings; assign batting positions via window ranking; map deliveries to phases; aggregate role-based strike rates per format and phase.

Key tables: Delivery, Innings, Match, Player, Team, Format.

Output fields: format_name, batting_role, match_phase, total_runs, total_balls, strike_rate.

SQL Query:

```
WITH batter_entry AS (  
    SELECT i.innings_id, d.striker_id AS player_id, MIN(d.over_number * 6 +  
d.ball_number) AS first_ball  
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id GROUP BY 1, 2  
) ,  
batting_position AS (  
    SELECT innings_id, player_id, ROW_NUMBER() OVER (PARTITION BY innings_id ORDER BY  
first_ball) AS batting_position FROM batter_entry
```

```

),
phase_runs AS (
    SELECT f.format_name, p.player_id, p.player_name, t.team_name, bp.batting_position,
    CASE WHEN d.over_number <= 6 AND f.format_name = 'T20' THEN 'Powerplay'
    WHEN d.over_number <= 10 AND f.format_name = 'ODI' THEN 'Powerplay'
    WHEN (f.format_name = 'T20' AND d.over_number BETWEEN 7 AND 16) OR
(f.format_name = 'ODI' AND d.over_number BETWEEN 11 AND 40) THEN 'Middle Overs'
    ELSE 'Death Overs' END AS match_phase,
    SUM(d.runs_batsman) AS runs_scored, COUNT(*) AS balls_faced
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id JOIN `Match` m ON
i.match_id = m.match_id
    JOIN Format f ON m.format_id = f.format_id JOIN Player p ON d.striker_id =
p.player_id
    JOIN Team t ON p.team_id = t.team_id JOIN batting_position bp ON bp.innings_id =
i.innings_id AND bp.player_id = p.player_id
    GROUP BY 1, 2, 3, 4, 5, 6
),
role_phase_sr AS (
    SELECT format_name, CASE WHEN batting_position BETWEEN 1 AND 3 THEN 'Top Order'
WHEN batting_position BETWEEN 4 AND 6 THEN 'Middle Order' END AS batting_role,
    match_phase, SUM(runs_scored) AS total_runs, SUM(balls_faced) AS total_balls,
ROUND(SUM(runs_scored) * 100.0 / SUM(balls_faced), 2) AS strike_rate
    FROM phase_runs WHERE batting_position BETWEEN 1 AND 6 GROUP BY 1, 2, 3
)
SELECT format_name, batting_role, match_phase, total_runs, total_balls, strike_rate
FROM role_phase_sr
ORDER BY format_name, CASE match_phase WHEN 'Powerplay' THEN 1 WHEN 'Middle Overs'
THEN 2 WHEN 'Death Overs' THEN 3 END,
CASE batting_role WHEN 'Top Order' THEN 1 WHEN 'Middle Order' THEN 2 END;

```

Output:

format_name	batting_role	match_phase	total_runs	total_balls	strike_rate
ODI	Top Order	Powerplay	15425	19899	77.52
ODI	Middle Order	Powerplay	1359	2500	54.36
ODI	Top Order	Middle Overs	19577	21726	90.11
ODI	Middle Order	Middle Overs	21173	25659	82.52
ODI	Top Order	Death Overs	1245	860	144.77
ODI	Middle Order	Death Overs	6046	4388	137.78
T20	Top Order	Powerplay	18094	17025	106.28
T20	Middle Order	Powerplay	2416	2949	81.93
T20	Top Order	Middle Overs	11952	9423	126.84
T20	Middle Order	Middle Overs	15192	13122	115.78
T20	Top Order	Death Overs	1123	624	179.97
T20	Middle Order	Death Overs	4013	2545	157.68

12 rows in set (4.43 sec)

Short analysis:

Top-order batters display controlled strike rates early, followed by acceleration in later phases, particularly in T20 matches. Middle-order batters show more stable strike rates across phases, prioritising consolidation over peak acceleration. This reflects role-based responsibility rather than inefficiency.

Q4 – Powerplay Impact on Match Outcome (Winning Margin)

Research aim: Test whether winners rely more heavily on powerplay scoring than losers.

Logic summary: Compute delivery-level total runs (bat + extras); aggregate powerplay runs and total runs per innings; compute powerplay share; label each innings as Win/Loss based on match winner; compare averages.

Key tables: Delivery, DeliveryExtra, Innings, Match, Team, Format.

Output fields: format_name, team_name, wins, PP_run_share_wins(%), losses, PP_run_share_losses(%).

SQL Query:

```
WITH team_ball_runs AS (
    SELECT f.format_name, m.match_id, t.team_id, t.team_name, d.over_number,
           d.runs_batsman + COALESCE(SUM(de.extra_runs), 0) AS runs_scored
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id
    JOIN Team t ON i.batting_team_id = t.team_id JOIN `Match` m ON i.match_id =
m.match_id
    JOIN Format f ON m.format_id = f.format_id LEFT JOIN DeliveryExtra de ON
d.delivery_id = de.delivery_id
    GROUP BY f.format_name, m.match_id, t.team_id, t.team_name, d.delivery_id,
d.over_number, d.runs_batsman
),
team_phase_runs AS (
    SELECT format_name, match_id, team_id, team_name, SUM(runs_scored) AS total_runs,
           SUM(CASE WHEN format_name = 'ODI' AND over_number BETWEEN 1 AND 10 THEN runs_scored
                    WHEN format_name = 'T20' AND over_number BETWEEN 1 AND 6 THEN runs_scored ELSE 0
           END) AS powerplay_runs
    FROM team_ball_runs GROUP BY format_name, match_id, team_id, team_name
),
team_powerplay_share AS (
    SELECT tpr.format_name, tpr.team_name, CASE WHEN m.winner_team_id = tpr.team_id
    THEN 'Win' ELSE 'Loss' END AS match_outcome,
           tpr.powerplay_runs / tpr.total_runs AS powerplay_share
    FROM team_phase_runs tpr JOIN `Match` m ON tpr.match_id = m.match_id WHERE
tpr.total_runs > 0
)
```

```

SELECT format_name, team_name, COUNT(CASE WHEN match_outcome = 'Win' THEN 1 END) AS
wins,
ROUND(AVG(CASE WHEN match_outcome = 'Win' THEN powerplay_share END) * 100, 2) AS
`PP_run_share_wins (%)`,
COUNT(CASE WHEN match_outcome = 'Loss' THEN 1 END) AS losses,
ROUND(AVG(CASE WHEN match_outcome = 'Loss' THEN powerplay_share END) * 100, 2) AS
`PP_run_share_losses (%)`
FROM team_powerplay_share GROUP BY format_name, team_name ORDER BY format_name,
`PP_run_share_wins (%)` DESC;

```

Output:

format_name	team_name	wins	PP_run_share_wins (%)	losses	PP_run_share_losses (%)
ODI	West Indies	7	36.30	14	21.77
ODI	New Zealand	21	30.70	13	19.65
ODI	Sri Lanka	13	23.64	16	26.64
ODI	Bangladesh	8	21.85	18	24.75
ODI	India	29	21.46	6	20.87
ODI	England	13	21.34	17	21.82
ODI	Australia	24	21.32	8	20.96
ODI	Pakistan	18	21.21	12	21.68
ODI	Canada	1	20.10	5	20.06
ODI	Netherlands	2	18.85	12	24.59
ODI	South Africa	18	18.06	14	25.79
ODI	Ireland	5	17.13	7	20.63
ODI	Zimbabwe	3	15.13	9	19.67
ODI	United Arab Emirates	0	NULL	6	17.93
ODI	Scotland	0	NULL	5	22.29
ODI	Kenya	0	NULL	6	28.80
T20	Netherlands	9	39.88	15	35.56
T20	England	22	39.14	16	31.76
T20	Scotland	7	38.58	9	29.81
T20	Ireland	6	36.85	16	37.56
T20	Australia	26	36.79	11	29.70
T20	New Zealand	19	36.08	14	28.28
T20	Sri Lanka	21	35.01	19	35.05
T20	Uganda	1	34.62	2	36.06
T20	South Africa	20	33.37	15	30.61
T20	Oman	2	31.64	7	38.31
T20	India	25	31.22	11	29.16
T20	Hong Kong	1	29.82	3	32.25
T20	West Indies	20	29.45	15	31.84
T20	Bangladesh	10	28.15	26	32.48
T20	Nepal	1	26.85	4	27.44
T20	Canada	1	26.28	2	29.02
T20	Pakistan	19	26.18	17	26.98
T20	Namibia	4	25.44	10	29.40
T20	Zimbabwe	7	24.95	10	41.51
T20	United States of America	1	20.81	5	29.62
T20	United Arab Emirates	1	20.27	5	29.70
T20	Papua New Guinea	0	NULL	6	28.24

38 rows in set (2.18 sec)

Short analysis:

In ODIs, excessive reliance on powerplay scoring does not strongly distinguish winners from losers, indicating the importance of balanced innings construction. In T20s, winning teams tend to score a higher proportion of runs during the powerplay, although over-dependence remains associated with losses.

Q5 – Bowling Effectiveness by Phase (Phase Specialists)

Research aim: Identify bowlers who take wickets most efficiently in Powerplay and Death overs.

Logic summary: Filter deliveries to high-impact overs by format; join wickets to deliveries; exclude run-outs from bowler credit; compute balls per wicket with minimum match filters; rank specialists by wickets then efficiency.

Key tables: Delivery, Innings, Match, Player, Wicket, MatchTeam, Team, Format.

Output fields: format_name, match_phase, bowler_name, team_name, matches_played, wickets, balls_bowled, balls_per_wicket.

SQL Query:

```
WITH phase_deliveries AS (
    SELECT f.format_name, m.match_id, p.player_id AS bowler_id, p.player_name AS
    bowler_name, t.team_name,
        CASE WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 1 AND 10 THEN
'Powerplay'
            WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 41 AND 50 THEN 'Death
Overs'
            WHEN f.format_name = 'T20' AND d.over_number BETWEEN 1 AND 6 THEN
'Powerplay'
            WHEN f.format_name = 'T20' AND d.over_number BETWEEN 16 AND 20 THEN 'Death
Overs' END AS match_phase,
        w.wicket_id, w.dismissal_type
    FROM Delivery d
    JOIN Innings i ON d.innings_id = i.innings_id
    JOIN `Match` m ON i.match_id = m.match_id
    JOIN Format f ON m.format_id = f.format_id
    JOIN Player p ON d.bowler_id = p.player_id
    JOIN MatchTeam mt ON mt.match_id = m.match_id AND mt.team_id <> i.batting_team_id
    JOIN Team t ON mt.team_id = t.team_id
    LEFT JOIN Wicket w ON d.delivery_id = w.delivery_id
    WHERE ((f.format_name = 'ODI' AND (d.over_number BETWEEN 1 AND 10 OR d.over_number
BETWEEN 41 AND 50))
        OR (f.format_name = 'T20' AND (d.over_number BETWEEN 1 AND 6 OR d.over_number
BETWEEN 16 AND 20)))
),
bowler_phase_stats AS (
    SELECT format_name, match_phase, bowler_id, bowler_name, team_name,
        COUNT(*) AS balls_bowled, COUNT(DISTINCT match_id) AS matches_played,
        SUM(CASE WHEN wicket_id IS NOT NULL AND dismissal_type <> 'run out' THEN 1 ELSE
0 END) AS wickets
```



```

FROM phase_deliveries WHERE match_phase IS NOT NULL
GROUP BY format_name, match_phase, bowler_id, bowler_name, team_name
),
filtered_bowler_stats AS (
  SELECT *, ROUND(balls_bowled / NULLIF(wickets, 0), 2) AS balls_per_wicket
  FROM bowler_phase_stats WHERE wickets > 0 AND matches_played >= 10
),
ranked_specialists AS (
  SELECT format_name, match_phase, bowler_name, team_name, matches_played, wickets,
balls_bowled, balls_per_wicket,
  ROW_NUMBER() OVER (PARTITION BY format_name, match_phase ORDER BY wickets DESC,
balls_per_wicket ASC) AS phase_rank
  FROM filtered_bowler_stats
)
SELECT format_name, match_phase, bowler_name, team_name, matches_played, wickets,
balls_bowled, balls_per_wicket
FROM ranked_specialists WHERE phase_rank <= 5
ORDER BY format_name, CASE match_phase WHEN 'Powerplay' THEN 1 WHEN 'Death Overs' THEN
2 END, phase_rank;

```

Output:

format_name	match_phase	bowler_name	team_name	matches_played	wickets	balls_bowled	balls_per_wicket
ODI	Powerplay	TA Boult	New Zealand	26	21	685	32.62
ODI	Powerplay	Mohammed Shami	India	17	17	354	20.82
ODI	Powerplay	CR Woakes	England	21	15	458	30.53
ODI	Powerplay	MA Starc	Australia	25	14	430	30.71
ODI	Powerplay	TG Southee	New Zealand	21	14	447	31.93
ODI	Death Overs	MA Starc	Australia	18	21	223	10.62
ODI	Death Overs	Wahab Riaz	Pakistan	12	16	204	12.75
ODI	Death Overs	Mustafizur Rahman	Bangladesh	11	14	194	13.86
ODI	Death Overs	JJ Bumrah	India	12	13	228	16.92
ODI	Death Overs	CR Woakes	England	13	12	149	12.42
T20	Powerplay	PA van Meekeren	Netherlands	15	14	158	11.29
T20	Powerplay	TG Southee	New Zealand	24	14	287	20.50
T20	Powerplay	JR Hazlewood	Australia	16	12	208	17.33
T20	Powerplay	K Rabada	South Africa	18	11	195	17.73
T20	Powerplay	TA Boult	New Zealand	16	10	161	16.10
T20	Death Overs	CJ Jordan	England	17	24	179	7.46
T20	Death Overs	Saeed Ajeel	Pakistan	16	16	150	9.38
T20	Death Overs	Mustafizur Rahman	Bangladesh	17	15	149	9.93
T20	Death Overs	TG Southee	New Zealand	20	15	167	11.13
T20	Death Overs	BP de Leede	Netherlands	10	13	86	6.62

Short analysis:

Powerplay wickets are primarily taken by specialist new-ball bowlers, particularly in ODIs, where early breakthroughs shape innings trajectories. Death-over wickets are dominated by bowlers with strong control and variation, especially in T20 formats. Specialisation by phase is more common than all-phase dominance.

Q6 – Dismissal Type Distribution (Wicket Mechanisms)

Research aim: Compare dismissal mechanisms across ODI and T20 and observe how wicket types shift by phase.

Logic summary: Link wickets to delivery overs; map each wicket to a phase; aggregate dismissal counts; use window functions to compute percentage share per phase and format.

Key tables: Wicket, Delivery, Innings, Match, Format.

Output fields: format_name, match_phase, dismissal_type, wickets, percentage_share.

SQL Query:

```
WITH phase_wickets AS (
    SELECT f.format_name,
    CASE WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 1 AND 10 THEN 'Powerplay'
         WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 11 AND 40 THEN 'Middle
Overs'
         WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 41 AND 50 THEN 'Death
Overs'
         WHEN f.format_name = 'T20' AND d.over_number BETWEEN 1 AND 6 THEN 'Powerplay'
         WHEN f.format_name = 'T20' AND d.over_number BETWEEN 7 AND 15 THEN 'Middle
Overs'
         WHEN f.format_name = 'T20' AND d.over_number BETWEEN 16 AND 20 THEN 'Death
Overs' END AS match_phase,
    w.dismissal_type
    FROM Wicket w JOIN Delivery d ON w.delivery_id = d.delivery_id
    JOIN Innings i ON d.innings_id = i.innings_id JOIN `Match` m ON i.match_id =
m.match_id
    JOIN Format f ON m.format_id = f.format_id
),
dismissal_counts AS (
    SELECT format_name, match_phase, dismissal_type, COUNT(*) AS wickets
    FROM phase_wickets WHERE match_phase IS NOT NULL
    GROUP BY format_name, match_phase, dismissal_type
),
dismissal_distribution AS (
    SELECT format_name, match_phase, dismissal_type, wickets,
    ROUND(wickets * 100.0 / SUM(wickets) OVER (PARTITION BY format_name, match_phase),
2) AS percentage_share
    FROM dismissal_counts
)
SELECT format_name, match_phase, dismissal_type, wickets, percentage_share
FROM dismissal_distribution
```

```
ORDER BY format_name, CASE match_phase WHEN 'Powerplay' THEN 1 WHEN 'Middle Overs'
THEN 2 WHEN 'Death Overs' THEN 3 END, wickets DESC;
```

Output:

format_name	match_phase	dismissal_type	wickets	percentage_share
ODI	Powerplay	caught	293	62.47
ODI	Powerplay	bowled	75	15.99
ODI	Powerplay	lbw	61	13.01
ODI	Powerplay	run out	18	3.84
ODI	Powerplay	caught and bowled	14	2.99
ODI	Powerplay	stumped	8	1.71
ODI	Middle Overs	caught	731	59.29
ODI	Middle Overs	bowled	229	18.57
ODI	Middle Overs	lbw	136	11.03
ODI	Middle Overs	run out	65	5.27
ODI	Middle Overs	caught and bowled	35	2.84
ODI	Middle Overs	stumped	31	2.51
ODI	Middle Overs	hit wicket	3	0.24
ODI	Middle Overs	retired hurt	2	0.16
ODI	Middle Overs	timed out	1	0.08
ODI	Death Overs	caught	459	60.79
ODI	Death Overs	bowled	140	18.54
ODI	Death Overs	run out	69	9.14
ODI	Death Overs	lbw	50	6.62
ODI	Death Overs	caught and bowled	22	2.91
ODI	Death Overs	stumped	13	1.72
ODI	Death Overs	retired hurt	1	0.13
ODI	Death Overs	hit wicket	1	0.13
T20	Powerplay	caught	453	59.29
T20	Powerplay	bowled	159	20.81
T20	Powerplay	lbw	63	8.25
T20	Powerplay	run out	33	4.32
T20	Powerplay	caught and bowled	27	3.53
T20	Powerplay	stumped	24	3.14
T20	Powerplay	retired hurt	4	0.52
T20	Powerplay	retired out	1	0.13
T20	Middle Overs	caught	691	59.31
T20	Middle Overs	bowled	219	18.80
T20	Middle Overs	lbw	100	8.58
T20	Middle Overs	run out	72	6.18
T20	Middle Overs	stumped	51	4.38
T20	Middle Overs	caught and bowled	31	2.66
T20	Middle Overs	retired hurt	1	0.09
T20	Death Overs	caught	501	61.10
T20	Death Overs	bowled	149	18.17
T20	Death Overs	run out	99	12.07
T20	Death Overs	lbw	31	3.78
T20	Death Overs	stumped	21	2.56
T20	Death Overs	caught and bowled	17	2.07
T20	Death Overs	hit wicket	2	0.24

Short analysis:

Caught dismissals are the dominant wicket type across all formats and phases, reflecting sustained fielding pressure. T20 death overs show a higher proportion of run-outs, indicating increased forced errors during acceleration phases. ODI dismissal patterns are more evenly distributed due to longer innings horizons.

3.3.2 Questions Q7–Q15: Context, Outcomes, Pressure, and Efficiency

Q7 – Knockout Match Impact Players (Big-Game Performers)

Research aim: Identify players with the highest combined impact in knockout matches.

Logic summary: Filter knockout matches; compute batting runs and bowling wickets per match; normalise per-match contributions relative to match maximums; combine batting and bowling into a single impact score; rank top players and label role dominance.

Key tables: Match, Innings, Delivery, Wicket, Player, MatchTeam, Team.

Output fields: player_name, team_name, role, knockout_matches_played, total_impact_score.

SQL Query:

```
WITH knockout_matches AS (
    SELECT m.match_id FROM `Match` m WHERE m.match_stage IN ('Quarter Final', 'Semi
Final', 'Final')
),
batting_stats AS (
    SELECT km.match_id, d.striker_id AS player_id, t.team_name, SUM(d.runs_batsman) AS
runs_scored
    FROM knockout_matches km JOIN Innings i ON km.match_id = i.match_id
    JOIN Delivery d ON i.innings_id = d.innings_id JOIN Team t ON i.batting_team_id =
t.team_id
    GROUP BY km.match_id, d.striker_id, t.team_name
),
batting_normalised AS (
    SELECT match_id, player_id, team_name, runs_scored / MAX(runs_scored) OVER
(PARTITION BY match_id) AS batting_score
    FROM batting_stats
),
bowling_stats AS (
    SELECT km.match_id, d.bowler_id AS player_id, t.team_name, COUNT(*) AS
wickets_taken
    FROM knockout_matches km JOIN Innings i ON km.match_id = i.match_id
    JOIN Delivery d ON i.innings_id = d.innings_id JOIN Wicket w ON d.delivery_id =
w.delivery_id
    JOIN MatchTeam mt ON mt.match_id = km.match_id AND mt.team_id <> i.batting_team_id
    JOIN Team t ON mt.team_id = t.team_id WHERE w.dismissal_type <> 'run out'
    GROUP BY km.match_id, d.bowler_id, t.team_name
),
bowling_normalised AS (
    SELECT match_id, player_id, team_name, wickets_taken / MAX(wickets_taken) OVER
(PARTITION BY match_id) AS bowling_score
    FROM bowling_stats
),
match_impact AS (
    SELECT COALESCE(b.match_id, w.match_id) AS match_id, COALESCE(b.player_id,
w.player_id) AS player_id,
    COALESCE(b.team_name, w.team_name) AS team_name, COALESCE(b.batting_score, 0) AS
batting_score,
    COALESCE(w.bowling_score, 0) AS bowling_score
```

```

FROM batting_normalised b LEFT JOIN bowling_normalised w ON b.match_id = w.match_id
AND b.player_id = w.player_id
UNION ALL
SELECT w.match_id, w.player_id, w.team_name, 0 AS batting_score, w.bowling_score
FROM bowling_normalised w WHERE NOT EXISTS (SELECT 1 FROM batting_normalised b
WHERE b.match_id = w.match_id AND b.player_id = w.player_id)
),
player_impact AS (
SELECT p.player_name, mi.team_name, COUNT(DISTINCT mi.match_id) AS
knockout_matches_played,
ROUND(SUM(mi.batting_score), 2) AS total_batting_impact,
ROUND(SUM(mi.bowling_score), 2) AS total_bowling_impact,
ROUND(SUM(mi.batting_score + mi.bowling_score), 2) AS total_impact_score
FROM match_impact mi JOIN Player p ON mi.player_id = p.player_id GROUP BY
p.player_name, mi.team_name
),
final_ranked AS (
SELECT player_name, team_name, knockout_matches_played, total_batting_impact,
total_bowling_impact, total_impact_score,
CASE WHEN total_batting_impact / total_impact_score >= 0.70 THEN 'Batter'
WHEN total_bowling_impact / total_impact_score >= 0.70 THEN 'Bowler' ELSE
'All-rounder' END AS role
FROM player_impact
)
SELECT player_name, team_name, role, knockout_matches_played, total_impact_score
FROM final_ranked ORDER BY total_impact_score DESC LIMIT 10;

```

Output:

player_name	team_name	role	knockout_matches_played	total_impact_score
V Kohli	India	Batter	14	7.73
TM Dilshan	Sri Lanka	Batter	9	5.76
MA Starc	Australia	Bowler	8	5.36
KS Williamson	New Zealand	Batter	12	5.11
AD Mathews	Sri Lanka	All-rounder	8	5.01
SPD Smith	Australia	Batter	9	5.01
RG Sharma	India	Batter	11	4.66
TA Boult	New Zealand	Bowler	8	4.54
SL Malinga	Sri Lanka	Bowler	7	4.35
JC Buttler	England	Batter	7	3.79

10 rows in set (0.05 sec)

Short analysis:

In knockout matches, a small group of players consistently contribute disproportionately through either batting, bowling, or both. Batting impact tends to dominate overall influence, while bowlers provide high-impact but more situational contributions. This highlights the importance of reliable run production under elimination pressure.

Q8 – Fielding Contributions by Team (Defensive Pressure)

Research aim: Compare team fielding impact in knockout matches (catches and run-outs).

Logic summary: Build a base map of team participation in knockouts; isolate wicket events attributable to fielding; aggregate catches vs run-outs; compute per-match averages; rank teams by total fielding dismissals.

Key tables: Match, Innings, Delivery, Wicket, MatchTeam, Team, Format.

Output fields: format_name, team_name, matches_played, catches, run_outs, avg_catches_per_match, avg_runouts_per_match, total_fielding_dismissals.

SQL Query:

```
WITH knockout_matches AS (
    SELECT m.match_id, f.format_name FROM `Match` m JOIN Format f ON m.format_id =
f.format_id
    WHERE m.match_stage IN ('Quarter Final', 'Semi Final', 'Final')
),
team_knockout_matches AS (
    SELECT km.format_name, mt.team_id, t.team_name, km.match_id FROM knockout_matches
km
    JOIN MatchTeam mt ON km.match_id = mt.match_id JOIN Team t ON mt.team_id =
t.team_id
),
fielding_events AS (
    SELECT km.format_name, mt.team_id, km.match_id, w.dismissal_type FROM
knockout_matches km
    JOIN Innings i ON km.match_id = i.match_id JOIN Delivery d ON i.innings_id =
d.innings_id
    JOIN Wicket w ON d.delivery_id = w.delivery_id JOIN MatchTeam mt ON mt.match_id =
km.match_id AND mt.team_id <> i.batting_team_id
    WHERE w.dismissal_type IN ('caught', 'caught and bowled', 'run out')
),
team_fielding_aggregates AS (
    SELECT tkm.format_name, tkm.team_name, COUNT(DISTINCT tkm.match_id) AS
matches_played,
    SUM(CASE WHEN fe.dismissal_type IN ('caught', 'caught and bowled') THEN 1 ELSE 0
END) AS catches,
    SUM(CASE WHEN fe.dismissal_type = 'run out' THEN 1 ELSE 0 END) AS run_outs
    FROM team_knockout_matches tkm LEFT JOIN fielding_events fe ON tkm.match_id =
fe.match_id AND tkm.team_id = fe.team_id
    GROUP BY tkm.format_name, tkm.team_name
),
ranked_teams AS (
```

```

SELECT format_name, team_name, matches_played, COALESCE(catches, 0) AS catches,
COALESCE(run_outs, 0) AS run_outs,
ROUND(COALESCE(catches, 0) / matches_played, 2) AS avg_catches_per_match,
ROUND(COALESCE(run_outs, 0) / matches_played, 2) AS avg_runouts_per_match,
(COALESCE(catches, 0) + COALESCE(run_outs, 0)) AS total_fielding_dismissals,
ROW_NUMBER() OVER (PARTITION BY format_name ORDER BY (COALESCE(catches, 0) +
COALESCE(run_outs, 0)) DESC) AS team_rank
FROM team_fielding_aggregates
)
SELECT format_name, team_name, matches_played, catches, run_outs,
avg_catches_per_match, avg_runouts_per_match, total_fielding_dismissals
FROM ranked_teams WHERE team_rank <= 5 ORDER BY format_name, total_fielding_dismissals
DESC;

```

Output:

format_name	team_name	matches_played	catches	run_outs	avg_catches_per_match	avg_runouts_per_match	total_fielding_dismissals
ODI	India	8	46	3	5.75	0.38	49
ODI	Australia	7	39	5	5.57	0.71	44
ODI	New Zealand	8	38	4	4.75	0.50	42
ODI	South Africa	4	28	1	5.00	0.25	21
ODI	England	3	9	2	3.00	0.67	11
T20	England	8	41	5	5.13	0.63	46
T20	West Indies	5	24	4	4.80	0.80	28
T20	Australia	5	16	2	3.20	0.40	18
T20	India	6	14	2	2.33	0.33	16
T20	Pakistan	5	14	1	2.80	0.20	15

10 rows in set (0.01 sec)

Short analysis:

Fielding contributions differ by format, with ODIs emphasising catching efficiency and T20s placing greater importance on run-outs. Teams that consistently convert fielding opportunities gain measurable advantages in knockout matches, reinforcing fielding as a strategic differentiator rather than a secondary skill.

Q9 – Team Scoring Progression (Format Pacing Patterns)

Research aim: Compare how team scoring rates evolve through match phases in ODI vs T20.

Logic summary: Identify top run-scoring teams per format; map deliveries to phases; aggregate runs and overs faced; compute run rate by phase and compare pacing patterns.

Key tables: Delivery, Innings, Match, Team, Format.

Output fields: format_name, team_name, match_phase, run_rate.

SQL Query:

```

WITH team_total_runs AS (
SELECT f.format_name, t.team_id, t.team_name, SUM(d.runs_total) AS total_runs
FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id

```

```

        JOIN `Match` m ON i.match_id = m.match_id JOIN Format f ON m.format_id =
f.format_id
        JOIN Team t ON i.batting_team_id = t.team_id GROUP BY 1, 2, 3
    ),
    ranked_teams AS (
        SELECT format_name, team_id, team_name, total_runs,
        ROW_NUMBER() OVER (PARTITION BY format_name ORDER BY total_runs DESC) AS team_rank
        FROM team_total_runs
    ),
    top_teams AS (
        SELECT format_name, team_id, team_name FROM ranked_teams WHERE team_rank <= 5
    ),
    phase_runs AS (
        SELECT f.format_name, t.team_name,
        CASE WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 1 AND 10 THEN 'Powerplay'
            WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 11 AND 40 THEN 'Middle
Overs'
            WHEN f.format_name = 'ODI' AND d.over_number BETWEEN 41 AND 50 THEN 'Death
Overs'
            WHEN f.format_name = 'T20' AND d.over_number BETWEEN 1 AND 6 THEN 'Powerplay'
            WHEN f.format_name = 'T20' AND d.over_number BETWEEN 7 AND 15 THEN 'Middle
Overs'
            WHEN f.format_name = 'T20' AND d.over_number BETWEEN 16 AND 20 THEN 'Death
Overs' END AS match_phase,
        SUM(d.runs_total) AS phase_runs, COUNT(DISTINCT CONCAT(i.innings_id, '-',
d.over_number)) AS overs_faced
        FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id JOIN `Match` m ON
i.match_id = m.match_id
        JOIN Format f ON m.format_id = f.format_id JOIN Team t ON i.batting_team_id =
t.team_id
        JOIN top_teams tt ON tt.team_id = t.team_id AND tt.format_name = f.format_name
GROUP BY 1, 2, 3
    )
SELECT format_name, team_name, match_phase, ROUND(phase_runs / overs_faced, 2) AS
run_rate
FROM phase_runs WHERE match_phase IS NOT NULL
ORDER BY format_name, team_name, CASE match_phase WHEN 'Powerplay' THEN 1 WHEN 'Middle
Overs' THEN 2 WHEN 'Death Overs' THEN 3 END;

```


Output:

format_name	team_name	match_phase	run_rate
ODI	Australia	Powerplay	5.37
ODI	Australia	Middle Overs	5.76
ODI	Australia	Death Overs	8.24
ODI	England	Powerplay	5.42
ODI	England	Middle Overs	5.47
ODI	England	Death Overs	7.70
ODI	India	Powerplay	5.57
ODI	India	Middle Overs	5.62
ODI	India	Death Overs	7.88
ODI	New Zealand	Powerplay	5.43
ODI	New Zealand	Middle Overs	5.39
ODI	New Zealand	Death Overs	8.52
ODI	South Africa	Powerplay	4.66
ODI	South Africa	Middle Overs	5.64
ODI	South Africa	Death Overs	9.91
T20	Australia	Powerplay	8.04
T20	Australia	Middle Overs	8.40
T20	Australia	Death Overs	9.86
T20	England	Powerplay	8.47
T20	England	Middle Overs	8.09
T20	England	Death Overs	9.91
T20	India	Powerplay	7.27
T20	India	Middle Overs	7.73
T20	India	Death Overs	10.10
T20	Pakistan	Powerplay	6.57
T20	Pakistan	Middle Overs	8.04
T20	Pakistan	Death Overs	9.13
T20	Sri Lanka	Powerplay	7.31
T20	Sri Lanka	Middle Overs	7.30
T20	Sri Lanka	Death Overs	8.75

30 rows in set (0.61 sec)

Short analysis:

ODI teams typically exhibit gradual run-rate acceleration across phases, preserving wickets for late-innings surges. T20 teams operate with flatter or front-loaded scoring profiles, reflecting early aggression. These differences highlight structural contrasts in pacing strategies between formats.

Q10 – Venue Scoring Patterns (Bias Matrix)

Research aim: Determine whether venues show systematic scoring differences between first and second innings.

Logic summary: Standardise venue names; aggregate innings totals per match; compute average 1st vs 2nd innings runs per venue by format; include match counts for reliability.

Key tables: Delivery, Innings, Match, Venue, Format.

Output fields: format_name, venue_name, venue_country, avg_1st_innings_run, avg_2nd_innings_run, matches_played.

SQL Query:

```
WITH innings_scores AS (  
    SELECT f.format_name, TRIM(SUBSTRING_INDEX(v.venue_name, ',', 1)) AS  
canonical_venue,
```

```

v.venue_country, i.innings_number, m.match_id, SUM(d.runs_total) AS innings_runs
FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id
JOIN `Match` m ON i.match_id = m.match_id JOIN Format f ON m.format_id =
f.format_id
JOIN Venue v ON m.venue_id = v.venue_id WHERE i.innings_number IN (1, 2)
GROUP BY f.format_name, canonical_venue, v.venue_country, i.innings_number,
m.match_id
),
venue_innings_avg AS (
SELECT format_name, canonical_venue AS venue_name,
GROUP_CONCAT(DISTINCT venue_country ORDER BY venue_country SEPARATOR ' / ') AS
venue_country,
AVG(CASE WHEN innings_number = 1 THEN innings_runs END) AS avg_1st_innings_run,
AVG(CASE WHEN innings_number = 2 THEN innings_runs END) AS avg_2nd_innings_run,
COUNT(DISTINCT match_id) AS matches_played
FROM innings_scores GROUP BY format_name, canonical_venue
)
SELECT format_name, venue_name, venue_country, ROUND(avg_1st_innings_run, 2) AS
avg_1st_innings_run,
ROUND(avg_2nd_innings_run, 2) AS avg_2nd_innings_run, matches_played
FROM venue_innings_avg ORDER BY format_name, venue_name;

```

Output:

format_name	venue_name	venue_country	avg_1st_innings_run	avg_2nd_innings_run	matches_played
ODI	Adelaide Oval	Unknown	256.25	230.25	4
ODI	Arun Jaitley Stadium	Delhi	368.67	232.67	3
ODI	Bellerive Oval	Hobart	274.67	234.67	3
ODI	Bharat Ratna Shri Atal Bihari Vajpayee Ekana Cricket Stadium	Lucknow	250.75	236.00	4
ODI	Brisbane Cricket Ground	Brisbane	256.50	247.00	2
ODI	Eden Gardens	Kolkata	274.25	285.50	4
ODI	Eden Park	Auckland	230.25	235.25	4
ODI	Edgbaston	Birmingham	270.00	260.00	5
ODI	Pervez Shah Kotla	Delhi	234.75	282.00	4
ODI	Hagley Oval	Christchurch	314.67	192.33	3
ODI	Headingley	Leeds	240.00	230.50	2
ODI	Himachal Pradesh Cricket Association Stadium	Sharamala	317.50	272.75	4
ODI	Kennington Oval	London	314.20	265.40	5
ODI	Lord's	London	270.40	259.80	5
ODI	M Chinnaswamy Stadium	Bangalore / Bengaluru	291.20	244.00	10
ODI	MA Chidambaram Stadium	Chennai	280.20	295.71	7
ODI	Maharashtra Cricket Association Stadium	Pune	314.50	228.50	4
ODI	Mahinda Rajapaksa International Cricket Stadium	Hambantota	324.50	117.00	2
ODI	Manuka Oval	Canberra	301.50	240.50	2
ODI	McLean Park	Nagpur	257.00	193.00	2
ODI	Melbourne Cricket Ground	Unknown	293.20	265.40	5
ODI	Narendra Modi Stadium	Ahmedabad	249.75	242.25	4
ODI	Old Trafford	Manchester	291.00	235.40	5
ODI	Pallekele International Cricket Stadium	Unknown	260.00	181.33	3
ODI	Punjab Cricket Association Stadium	Chandigarh	295.33	194.00	3
ODI	R Premadasa Stadium	Colombo	195.00	196.50	7
ODI	Rajiv Gandhi International Stadium	Hyderabad	317.33	257.67	3
ODI	Riverside Ground	Chencher-le-Street	262.00	225.67	3
ODI	Sardar Patel Stadium	Ahmedabad	228.00	199.33	3
ODI	Santon Oval	Nelson	302.33	305.00	3
ODI	Seddon Park	Hamilton	295.33	275.67	3
ODI	Shere Bangla National Stadium	Mirpur	200.33	147.17	6
ODI	Sigma Gardens	Cardiff	261.00	200.50	2
ODI	Sydney Cricket Ground	Unknown	311.25	207.50	4
ODI	The Cooper Associates County Ground	Taunton	314.00	294.00	2
ODI	The Rose Bowl	Southampton	156.00	221.50	3
ODI	Trent Bridge	Nottingham	280.50	262.00	4
ODI	University Oval	Dunedin	142.00	146.00	1
ODI	Visakhapatnam Cricket Association Stadium	Nagpur	271.00	231.50	4
ODI	Wankhede Stadium	Mumbai	340.43	228.46	7
ODI	Western Australia Cricket Association Ground	Perth	142.00	144.50	2
ODI	Westpac Stadium	Wellington	291.50	220.50	4
ODI	Zohur Ahmed Chowdhury Stadium	Chittagong	192.50	196.50	2
T20	Adelaide Oval	Adelaide	150.50	143.00	6
T20	Al Amerat Cricket Ground Oman Cricket (Ministry Turf 1)	Al Amerat	140.33	126.67	6
T20	Arncliffe Vale Ground	Kingstown	120.67	111.00	3
T20	Beausejour Stadium	St Lucia	150.22	152.70	9
T20	Bellerive Oval	Hobart	143.50	129.44	9
T20	Brian Lara Stadium	Trinidad	85.00	85.33	3
T20	Brisbane Cricket Ground	Brisbane	169.33	147.67	3
T20	Central Broward Regional Park Stadium Turf Ground	Lauderhill	100.00	111.00	1
T20	Garen Sammy National Cricket Stadium	Gros Islet	105.00	104.40	5
T20	Dubai International Cricket Stadium	Dubai	129.00	129.42	12
T20	Eden Gardens	Kolkata	154.75	124.00	4
T20	Pervez Shah Kotla	Delhi	140.00	147.33	3
T20	QEHK Stadium	Geelong	142.00	117.00	6
T20	Grand Prairie Stadium	Dallas	145.75	147.50	4
T20	Himachal Pradesh Cricket Association Stadium	Sharamala	130.33	100.00	6
T20	Kensington Oval	Barbados / Bridgetown	153.33	136.35	10
T20	M Chinnaswamy Stadium	Bangalore	143.33	143.00	3
T20	Mahinda Rajapaksa International Cricket Stadium	Hambantota	117.67	80.00	3
T20	Melbourne Cricket Ground	Melbourne	159.75	129.50	4
T20	Nassau County International Cricket Stadium	New York	107.63	100.00	8
T20	Pallekele International Cricket Stadium	Unknown	104.50	153.33	9
T20	Perth Stadium	Perth	127.75	129.75	4
T20	Providence Stadium	Guyana / Providence	130.00	70.30	10
T20	Punjab Cricket Association IS Bindra Stadium	Chandigarh	177.67	103.67	3
T20	R Premadasa Stadium	Colombo	140.30	123.42	13
T20	Sharjah Cricket Stadium	Sharjah	140.20	130.70	10
T20	Shere Bangla National Stadium	Mirpur	150.00	131.54	13
T20	Sir Vivian Richards Stadium	North Sound	132.00	113.30	8
T20	Sydney Cricket Ground	Sydney	175.57	120.70	7
T20	Sydney Stadium	Unknown	147.00	140.00	4
T20	Visakhapatnam Cricket Association Stadium	Nagpur	130.00	112.00	5
T20	Wankhede Stadium	Mumbai	201.00	203.00	3
T20	Zohur Ahmed Chowdhury Stadium	Chittagong / Chittagong	143.25	119.77	13
T20	Zayed Cricket Stadium	Abu Dhabi	139.42	130.25	12

77 rows in set (0.77 sec)

Short analysis:

Several venues display consistent differences between first- and second-innings scores, particularly in ODIs where pitch conditions evolve over time. T20 venues show higher volatility and smaller innings gaps. Match volume helps distinguish structural venue effects from random variation.

Q11 – Death-Over Bowling Economy (Pressure Control)

Research aim: Identify bowlers who consistently restrict scoring in death overs across formats.

Logic summary: Filter deliveries to death overs by format; aggregate runs conceded and balls bowled; exclude run-outs from wickets; compute economy rate and apply minimum match filters; rank by economy then wickets.

Key tables: Delivery, Innings, Match, Player, Wicket, MatchTeam, Team, Format.

Output fields: format_name, bowler_name, team_name, matches_played, runs_conceded, balls_bowled, wickets, death_eco_rate.

SQL Query:

```
WITH death_over_deliveries AS (
    SELECT f.format_name, m.match_id, p.player_id AS bowler_id, p.player_name AS
    bowler_name, t.team_name,
    d.runs_total, w.wicket_id, w.dismissal_type
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id JOIN `Match` m ON
    i.match_id = m.match_id
    JOIN Format f ON m.format_id = f.format_id JOIN Player p ON d.bowler_id =
    p.player_id
    JOIN MatchTeam mt ON mt.match_id = m.match_id AND mt.team_id <> i.batting_team_id
    JOIN Team t ON mt.team_id = t.team_id LEFT JOIN Wicket w ON d.delivery_id =
    w.delivery_id
    WHERE ((f.format_name = 'ODI' AND d.over_number BETWEEN 41 AND 50)
    OR (f.format_name = 'T20' AND d.over_number BETWEEN 16 AND 20))
),
bowler_death_stats AS (
    SELECT format_name, bowler_id, bowler_name, team_name, COUNT(*) AS balls_bowled,
    SUM(runs_total) AS runs_conceded,
    COUNT(DISTINCT match_id) AS matches_played,
    SUM(CASE WHEN wicket_id IS NOT NULL AND dismissal_type <> 'run out' THEN 1 ELSE 0
    END) AS wickets
    FROM death_over_deliveries GROUP BY format_name, bowler_id, bowler_name, team_name
),
filtered_bowlers AS (
    SELECT *, ROUND((runs_conceded * 6.0) / balls_bowled, 2) AS death_eco_rate
    FROM bowler_death_stats WHERE matches_played >= 10 AND balls_bowled > 0 AND
    (runs_conceded * 6.0) / balls_bowled < 10
),
ranked_bowlers AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY format_name ORDER BY death_eco_rate ASC,
    wickets DESC) AS format_rank
    FROM filtered_bowlers
```

```

)
SELECT format_name, bowler_name, team_name, matches_played, runs_conceded,
balls_bowled, wickets, death_eco_rate
FROM ranked_bowlers WHERE format_rank <= 10 ORDER BY format_name, death_eco_rate ASC,
wickets DESC;

```

Output:

format_name	bowler_name	team_name	matches_played	runs_conceded	balls_bowled	wickets	death_eco_rate
ODI	JJ Bumrah	India	12	199	220	13	5.43
ODI	MA Starc	Australia	18	206	223	21	5.54
ODI	Ishan Tahir	South Africa	13	118	126	7	5.62
ODI	MJ Santner	New Zealand	11	95	98	5	5.82
ODI	RA Jadeja	India	10	85	85	5	6.00
ODI	MA Wood	England	13	209	208	11	6.03
ODI	SL Malinga	Sri Lanka	13	195	193	12	6.06
ODI	Mahab Riaz	Pakistan	12	215	204	16	6.32
ODI	TG Southee	New Zealand	12	182	168	11	6.50
ODI	MJ Henry	New Zealand	11	118	106	9	6.68
T20	JJ Bumrah	India	13	94	99	8	5.70
T20	TA Boult	New Zealand	15	104	104	10	6.00
T20	MRJ Watt	Scotland	12	85	75	6	6.80
T20	Mustafizur Rahman	Bangladesh	17	171	149	15	6.89
T20	Naris Rauf	Pakistan	13	143	115	10	7.46
T20	Taskin Ahmed	Bangladesh	10	90	70	11	7.71
T20	DN Steyn	South Africa	13	145	100	8	8.06
T20	A Nortje	South Africa	14	132	98	12	8.08
T20	Mohammad Amir	Pakistan	13	160	118	7	8.14
T20	K Rabada	South Africa	16	154	112	10	8.25

20 rows in set (0.19 sec)

Short analysis:

Bowling economy in death overs varies significantly across formats, with tighter margins in T20 matches. Bowlers who maintain low economy rates under pressure tend to rely on accuracy, variation, and situational awareness rather than pace alone. Consistency across matches is a stronger indicator than isolated performances.

Q12 – Top-Order Stability vs Win Rate

Research aim: Measure how winning probability changes with top-order stability (runs scored before the 2nd wicket).

Logic summary: Use window functions to compute running totals of runs and wickets per innings; extract runs before the 2nd wicket; convert into a percentage of innings total; categorise into stability bands; compute win rates per band.

Key tables: Delivery, Wicket, Innings, Match, Team, Format.

Output fields: format_name, team_name, top_order_contribution_band, matches_played, matches_won, win_percentage.

SQL Query:

```
WITH wicket_progression AS (
    SELECT i.match_id, i.batting_team_id, f.format_name, d.delivery_id,
    SUM(d.runs_total) OVER (PARTITION BY i.match_id, i.batting_team_id ORDER BY
d.delivery_id) AS cumulative_runs,
    COUNT(w.delivery_id) OVER (PARTITION BY i.match_id, i.batting_team_id ORDER BY
d.delivery_id) AS cumulative_wickets
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id JOIN `Match` m ON
i.match_id = m.match_id
    JOIN Format f ON m.format_id = f.format_id LEFT JOIN Wicket w ON d.delivery_id =
w.delivery_id
),
runs_upto_2nd_wicket AS (
    SELECT match_id, batting_team_id, format_name, MAX(cumulative_runs) AS
runs_before_2nd_wicket
    FROM wicket_progression WHERE cumulative_wickets < 2 GROUP BY 1, 2, 3
),
total_team_runs AS (
    SELECT i.match_id, i.batting_team_id, SUM(d.runs_total) AS total_runs
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id GROUP BY 1, 2
),
match_level_contribution AS (
    SELECT r.match_id, r.batting_team_id, r.format_name, (r.runs_before_2nd_wicket /
NULLIF(t.total_runs, 0)) * 100 AS pct_top_order_runs
    FROM runs_upto_2nd_wicket r JOIN total_team_runs t ON r.match_id = t.match_id AND
r.batting_team_id = t.batting_team_id
),
categorized_matches AS (
    SELECT match_id, batting_team_id, format_name,
    CASE WHEN pct_top_order_runs < 30 THEN 'Low (<30%)' WHEN pct_top_order_runs <= 60
THEN 'Moderate (30-60%)' ELSE 'High (>60%)' END AS top_order_contribution_band
    FROM match_level_contribution
)
SELECT cm.format_name, t.team_name, cm.top_order_contribution_band, COUNT(*) AS
matches_played,
SUM(CASE WHEN m.winner_team_id = cm.batting_team_id THEN 1 ELSE 0 END) AS matches_won,
ROUND(100.0 * SUM(CASE WHEN m.winner_team_id = cm.batting_team_id THEN 1 ELSE 0 END) /
COUNT(*), 2) AS win_percentage
FROM categorized_matches cm JOIN `Match` m ON cm.match_id = m.match_id JOIN Team t ON
cm.batting_team_id = t.team_id
GROUP BY 1, 2, 3 HAVING COUNT(*) >= 3 ORDER BY 1, 2, 3;
```

Output:

format_name	team_name	top_order_contribution_band	matches_played	matches_won	win_percentage
ODI	Australia	High (>60%)	8	8	100.00
ODI	Australia	Low (<30%)	13	10	76.92
ODI	Australia	Moderate (30-60%)	11	6	54.55
ODI	Bangladesh	Low (<30%)	18	4	22.22
ODI	Bangladesh	Moderate (30-60%)	8	4	50.00
ODI	Canada	Low (<30%)	5	1	20.00
ODI	England	High (>60%)	6	6	100.00
ODI	England	Low (<30%)	14	1	7.14
ODI	England	Moderate (30-60%)	10	6	60.00
ODI	India	High (>60%)	8	7	87.50
ODI	India	Low (<30%)	12	11	91.67
ODI	India	Moderate (30-60%)	15	11	73.33
ODI	Ireland	Low (<30%)	9	3	33.33
ODI	Ireland	Moderate (30-60%)	3	2	66.67
ODI	Nepal	Low (<30%)	4	0	0.00
ODI	Netherlands	Low (<30%)	11	2	18.18
ODI	Netherlands	Moderate (30-60%)	2	0	0.00
ODI	New Zealand	High (>60%)	6	5	83.33
ODI	New Zealand	Low (<30%)	19	9	47.37
ODI	New Zealand	Moderate (30-60%)	9	7	77.78
ODI	Pakistan	High (>60%)	4	4	100.00
ODI	Pakistan	Low (<30%)	14	8	57.14
ODI	Pakistan	Moderate (30-60%)	12	6	50.00
ODI	Scotland	Low (<30%)	5	0	0.00
ODI	South Africa	High (>60%)	5	4	80.00
ODI	South Africa	Low (<30%)	15	7	46.67
ODI	South Africa	Moderate (30-60%)	12	7	58.33
ODI	Sri Lanka	High (>60%)	8	6	75.00
ODI	Sri Lanka	Low (<30%)	13	5	38.46
ODI	Sri Lanka	Moderate (30-60%)	8	2	25.00
ODI	United Arab Emirates	Low (<30%)	6	0	0.00
ODI	West Indies	Low (<30%)	12	1	8.33
ODI	West Indies	Moderate (30-60%)	7	4	57.14
ODI	Zimbabwe	Low (<30%)	10	3	30.00
T20	Australia	High (>60%)	12	12	100.00
T20	Australia	Low (<30%)	16	8	50.00
T20	Australia	Moderate (30-60%)	9	6	66.67
T20	Bangladesh	Low (<30%)	22	6	27.27
T20	Bangladesh	Moderate (30-60%)	12	3	25.00
T20	England	High (>60%)	8	8	100.00
T20	England	Low (<30%)	13	5	38.46
T20	England	Moderate (30-60%)	17	9	52.94
T20	Hong Kong	Low (<30%)	3	0	0.00
T20	India	High (>60%)	6	5	83.33
T20	India	Low (<30%)	15	10	66.67
T20	India	Moderate (30-60%)	15	10	66.67
T20	Ireland	High (>60%)	5	4	80.00
T20	Ireland	Low (<30%)	11	1	9.09
T20	Ireland	Moderate (30-60%)	6	1	16.67
T20	Namibia	Low (<30%)	7	2	28.57
T20	Namibia	Moderate (30-60%)	6	2	33.33
T20	Nepal	Low (<30%)	3	1	33.33
T20	Netherlands	High (>60%)	5	5	100.00
T20	Netherlands	Low (<30%)	11	1	9.09
T20	Netherlands	Moderate (30-60%)	8	3	37.50
T20	New Zealand	High (>60%)	5	5	100.00
T20	New Zealand	Low (<30%)	11	6	54.55
T20	New Zealand	Moderate (30-60%)	17	8	47.06
T20	Oman	Low (<30%)	5	0	0.00
T20	Oman	Moderate (30-60%)	3	1	33.33
T20	Pakistan	High (>60%)	9	8	88.89
T20	Pakistan	Low (<30%)	13	5	38.46
T20	Pakistan	Moderate (30-60%)	14	6	42.86
T20	Papua New Guinea	Low (<30%)	6	0	0.00
T20	Scotland	High (>60%)	3	2	66.67
T20	Scotland	Low (<30%)	4	1	25.00
T20	Scotland	Moderate (30-60%)	9	4	44.44
T20	South Africa	High (>60%)	6	4	66.67
T20	South Africa	Low (<30%)	10	7	70.00
T20	South Africa	Moderate (30-60%)	11	9	81.82
T20	Sri Lanka	High (>60%)	10	8	80.00
T20	Sri Lanka	Low (<30%)	16	8	50.00
T20	Sri Lanka	Moderate (30-60%)	14	5	35.71
T20	Uganda	Low (<30%)	3	1	33.33
T20	United Arab Emirates	Low (<30%)	3	0	0.00
T20	United States of America	Moderate (30-60%)	4	0	0.00
T20	West Indies	High (>60%)	5	4	80.00
T20	West Indies	Low (<30%)	17	8	47.06
T20	West Indies	Moderate (30-60%)	13	8	61.54
T20	Zimbabwe	Low (<30%)	10	5	50.00
T20	Zimbabwe	Moderate (30-60%)	5	2	40.00

81 rows in set (2.19 sec)

Short analysis:

Higher top-order contribution correlates positively with winning in both formats, with a stronger effect observed in T20s. ODIs allow greater recovery following early wickets, whereas T20s offer limited scope for regrouping. Early batting stability therefore acts as a key success multiplier.

Q13 – Extras Contribution Analysis (Indiscipline Analysis)

Research aim: Quantify how much of total scoring comes from extras and which types dominate.

Logic summary: Aggregate extras by type via DeliveryExtra; join to total team runs; compute average extras per match and percentage contribution relative to total runs; filter for sufficient match samples.

Key tables: DeliveryExtra, Delivery, Innings, Match, Team, Format.

Output fields: format_name, team_name, matches_played, wide_runs, no_ball_runs, bye_runs, leg_bye_runs, total_extras, avg_extras_per_match, extras_pct_of_team_runs.

SQL Query:

```
WITH team_matches AS (
    SELECT f.format_name, t.team_id, t.team_name, COUNT(DISTINCT m.match_id) AS
matches_played
    FROM `Match` m JOIN Format f ON m.format_id = f.format_id
    JOIN MatchTeam mt ON m.match_id = mt.match_id JOIN Team t ON mt.team_id = t.team_id
    GROUP BY f.format_name, t.team_id, t.team_name
),
team_runs AS (
    SELECT f.format_name, t.team_id, SUM(d.runs_total) AS total_team_runs
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id
    JOIN `Match` m ON i.match_id = m.match_id JOIN Format f ON m.format_id =
f.format_id
    JOIN Team t ON i.batting_team_id = t.team_id GROUP BY f.format_name, t.team_id
),
team_extras AS (
    SELECT f.format_name, t.team_id,
    SUM(CASE WHEN de.extra_type = 'wides' THEN de.extra_runs ELSE 0 END) AS wide_runs,
    SUM(CASE WHEN de.extra_type = 'noballs' THEN de.extra_runs ELSE 0 END) AS
no_ball_runs,
    SUM(CASE WHEN de.extra_type = 'byes' THEN de.extra_runs ELSE 0 END) AS bye_runs,
    SUM(CASE WHEN de.extra_type = 'legbyes' THEN de.extra_runs ELSE 0 END) AS
leg_bye_runs,
    SUM(CASE WHEN de.extra_type IN ('wides', 'noballs', 'byes', 'legbyes') THEN
de.extra_runs ELSE 0 END) AS total_extras
    FROM DeliveryExtra de JOIN Delivery d ON de.delivery_id = d.delivery_id
    JOIN Innings i ON d.innings_id = i.innings_id JOIN `Match` m ON i.match_id =
m.match_id
    JOIN Format f ON m.format_id = f.format_id JOIN Team t ON i.batting_team_id =
t.team_id
    GROUP BY f.format_name, t.team_id
```



```

)

SELECT tm.format_name, tm.team_name, tm.matches_played,
COALESCE(te.wide_runs, 0) AS wide_runs, COALESCE(te.no_ball_runs, 0) AS no_ball_runs,
COALESCE(te.bye_runs, 0) AS bye_runs, COALESCE(te.leg_bye_runs, 0) AS leg_bye_runs,
COALESCE(te.total_extras, 0) AS total_extras,
ROUND(COALESCE(te.total_extras, 0) / tm.matches_played, 2) AS avg_extras_per_match,
ROUND(COALESCE(te.total_extras, 0) * 100.0 / tr.total_team_runs, 2) AS
extras_pct_of_team_runs
FROM team_matches tm
LEFT JOIN team_extras te ON tm.team_id = te.team_id AND tm.format_name =
te.format_name
LEFT JOIN team_runs tr ON tm.team_id = tr.team_id AND tm.format_name = tr.format_name
WHERE tm.matches_played >= 3 AND tr.total_team_runs > 0
ORDER BY tm.format_name, extras_pct_of_team_runs DESC;

```

Output:

format_name	team_name	matches_played	wide_runs	no_ball_runs	bye_runs	leg_bye_runs	total_extras	avg_extras_per_match	extras_pct_of_team_runs
ODI	Kenya	6	39	8	11	26	84	14.00	9.81
ODI	United Arab Emirates	6	63	5	9	35	112	18.67	9.99
ODI	Canada	6	63	4	6	18	91	15.17	8.63
ODI	West Indies	22	192	12	24	84	312	14.18	6.40
ODI	Netherlands	14	97	18	14	53	172	12.29	6.00
ODI	Bangladesh	26	224	14	14	113	365	14.04	5.84
ODI	Zimbabwe	12	98	8	11	62	179	14.92	5.78
ODI	New Zealand	34	225	27	38	128	468	13.76	5.62
ODI	Pakistan	38	224	23	14	79	360	9.47	5.47
ODI	Sri Lanka	29	225	23	65	75	388	13.38	5.45
ODI	Ireland	12	78	12	13	57	160	13.33	5.34
ODI	England	38	268	18	25	98	394	10.37	5.80
ODI	Australia	33	287	15	28	113	443	13.42	4.77
ODI	South Africa	32	245	22	27	93	387	12.09	4.58
ODI	India	25	235	38	38	98	409	16.36	4.41
ODI	Scotland	5	36	2	1	9	48	9.60	3.84
T20	Uganda	3	22	8	2	4	36	12.00	17.83
T20	Nepal	5	32	8	8	13	61	12.20	8.79
T20	United Arab Emirates	6	35	1	3	17	56	9.33	7.76
T20	Zimbabwe	17	77	8	12	68	157	9.24	7.64
T20	Papua New Guinea	6	17	3	4	25	49	8.17	7.37
T20	United States of America	6	52	5	9	6	63	10.50	6.98
T20	Namibia	14	58	9	16	38	121	8.64	6.89
T20	Ireland	23	85	6	22	69	182	7.91	6.75
T20	Oman	9	48	12	1	17	78	8.67	6.71
T20	West Indies	36	188	28	38	118	364	10.11	6.65
T20	Canada	3	17	8	1	18	34	11.33	6.43
T20	Netherlands	24	89	18	14	72	193	8.04	6.38
T20	Scotland	16	72	12	4	47	134	8.38	6.21
T20	Bangladesh	36	158	12	27	98	295	8.19	5.98
T20	New Zealand	13	141	14	27	89	271	20.85	5.88
T20	Sri Lanka	48	183	14	28	94	319	6.65	5.78
T20	Pakistan	36	187	17	37	88	309	8.58	5.56
T20	England	39	188	28	13	86	288	7.38	5.15
T20	Hong Kong	4	6	1	5	13	25	6.25	5.87
T20	South Africa	25	188	22	41	67	238	9.52	4.84
T20	Australia	17	141	29	15	77	262	15.41	4.78
T20	India	36	153	23	28	55	259	7.19	4.68

Short analysis:

Extras constitute a higher proportion of total runs in T20 matches compared to ODIs due to shorter innings lengths. Wides represent the largest component of extras across formats, indicating that bowling discipline remains a critical competitive factor.

Q14 – Chase Efficiency and Success (Mastery Matrix)

Research aim: Evaluate how effectively teams convert chases into wins and how efficiently they score relative to an expected baseline.

Logic summary: Isolate successful chases (2nd innings wins); calculate achieved run rate using legal balls; compute chase success % and a run-rate efficiency ratio; filter for meaningful sample sizes.

Key tables: Match, Innings, Delivery, DeliveryExtra, Team, Format.

Output fields: format_name, team_name, total_matches_played, chases_played, successful_chases, chase_success_pct, avg_required_rr, avg_achieved_rr, rr_efficiency_ratio.

SQL Query:

```
WITH chase_level AS (
    SELECT f.format_name, m.match_id, ia2.batting_team_id AS team_id,
    SUM(d.runs_batsman + IFNULL(de.extra_runs, 0)) AS runs_scored,
    COUNT(CASE WHEN de.extra_type IS NULL OR de.extra_type NOT IN ('wides', 'noballs')
    THEN 1 END) AS legal_balls
    FROM `Match` m JOIN Format f ON m.format_id = f.format_id
    JOIN Innings ia2 ON m.match_id = ia2.match_id AND ia2.innings_number = 2
    JOIN Delivery d ON d.innings_id = ia2.innings_id LEFT JOIN DeliveryExtra de ON
    d.delivery_id = de.delivery_id
    WHERE ia2.batting_team_id = m.winner_team_id GROUP BY 1, 2, 3
),
team_aggregate AS (
    SELECT format_name, team_id, COUNT(*) AS successful_chases, AVG((runs_scored /
legal_balls) * 6) AS avg_achieved_rr,
    AVG(CASE WHEN format_name = 'ODI' THEN (runs_scored / 300) * 6 WHEN format_name =
'T20' THEN (runs_scored / 120) * 6 END) AS avg_required_rr
    FROM chase_level GROUP BY 1, 2
),
chases_played AS (
    SELECT f.format_name, ia2.batting_team_id AS team_id, COUNT(DISTINCT m.match_id) AS
chases_played
    FROM `Match` m JOIN Format f ON m.format_id = f.format_id
    JOIN Innings ia2 ON m.match_id = ia2.match_id AND ia2.innings_number = 2 GROUP BY
1, 2
),
match_counts AS (
    SELECT f.format_name, i.batting_team_id AS team_id, COUNT(DISTINCT m.match_id) AS
total_matches_played
    FROM `Match` m JOIN Format f ON m.format_id = f.format_id
    JOIN Innings i ON m.match_id = i.match_id GROUP BY 1, 2
```

```

)
SELECT ta.format_name, t.team_name, mc.total_matches_played, cp.chases_played,
ta.successful_chases,
ROUND((ta.successful_chases / cp.chases_played) * 100, 2) AS chase_success_pct,
ROUND(ta.avg_required_rr, 2) AS avg_required_rr, ROUND(ta.avg_achieved_rr, 2) AS
avg_achieved_rr,
ROUND(ta.avg_achieved_rr / ta.avg_required_rr, 3) AS rr_efficiency_ratio
FROM team_aggregate ta JOIN chases_played cp ON ta.format_name = cp.format_name AND
ta.team_id = cp.team_id
JOIN match_counts mc ON ta.format_name = mc.format_name AND ta.team_id = mc.team_id
JOIN Team t ON ta.team_id = t.team_id WHERE ta.successful_chases >= 5
ORDER BY ta.format_name, chase_success_pct DESC, rr_efficiency_ratio DESC;

```

Output:

format_name	team_name	total_matches_played	chases_played	successful_chases	chase_success_pct	avg_required_rr	avg_achieved_rr	rr_efficiency_ratio
ODI	India	35	17	14	82.35	4.57	5.99	1.304
ODI	Australia	32	12	9	75.00	4.29	6.22	1.448
ODI	New Zealand	34	19	13	68.42	3.97	6.86	1.726
ODI	Sri Lanka	29	12	6	50.00	4.44	6.89	1.551
ODI	Pakistan	38	16	8	50.00	4.22	5.75	1.363
ODI	South Africa	32	14	5	35.71	4.54	5.99	1.320
ODI	Bangladesh	26	16	5	31.25	5.28	5.99	1.135
T20	India	36	15	12	80.00	6.58	8.21	1.247
T20	Australia	37	25	18	72.00	6.99	9.33	1.334
T20	West Indies	35	13	9	69.23	7.29	8.89	1.220
T20	England	38	27	17	62.96	7.12	9.48	1.333
T20	New Zealand	33	15	8	53.33	6.86	7.45	1.085
T20	Pakistan	36	17	9	52.94	6.64	7.35	1.106
T20	Sri Lanka	48	16	8	50.00	5.76	8.09	1.390
T20	South Africa	35	18	9	50.00	5.72	6.79	1.176
T20	Netherlands	24	13	6	46.15	6.73	7.34	1.090
T20	Ireland	22	13	5	38.46	7.84	8.38	1.069

Short analysis:

Successful run chases are characterised by higher achieved run rates relative to required run rates, particularly in T20 formats. Teams with consistent chase efficiency demonstrate superior tempo control and decision-making under pressure, while less successful teams struggle to maintain momentum.

Q15 – Momentum Impact After Wickets (Wicket Crash)

Research aim: Measure how wicket loss affects scoring momentum by comparing run rate before vs after dismissals.

Logic summary: Build a ball-index timeline; identify wicket events; compute runs and balls in a fixed window around each wicket (ODI 24 balls, T20 12 balls); calculate average RR before/after and net change by team and format.

Key tables: Wicket, Delivery, Innings, Match, Team, Format.

Output fields: format_name, team_name, avg_rr_before_wicket, avg_rr_after_wicket, avg_rr_change, wicket_events.

SQL Query:

```
WITH wicket_events AS (
    SELECT f.format_name, m.match_id, i.innings_id, i.batting_team_id AS team_id,
    d.delivery_id,
    d.over_number, d.ball_number, (d.over_number * 6 + d.ball_number) AS ball_index
    FROM Wicket w JOIN Delivery d ON w.delivery_id = d.delivery_id JOIN Innings i ON
    d.innings_id = i.innings_id
    JOIN `Match` m ON i.match_id = m.match_id JOIN Format f ON m.format_id =
    f.format_id
),
delivery_runs AS (
    SELECT i.innings_id, (d.over_number * 6 + d.ball_number) AS ball_index,
    (d.runs_batsman + IFNULL(de.extra_runs, 0)) AS runs_scored
    FROM Delivery d JOIN Innings i ON d.innings_id = i.innings_id LEFT JOIN
    DeliveryExtra de ON d.delivery_id = de.delivery_id
),
wicket_windows AS (
    SELECT we.format_name, we.team_id,
    SUM(CASE WHEN dr.ball_index BETWEEN we.ball_index - CASE WHEN we.format_name =
    'ODI' THEN 24 ELSE 12 END AND we.ball_index - 1 THEN dr.runs_scored END) AS
    runs_before,
    COUNT(CASE WHEN dr.ball_index BETWEEN we.ball_index - CASE WHEN we.format_name =
    'ODI' THEN 24 ELSE 12 END AND we.ball_index - 1 THEN 1 END) AS balls_before,
    SUM(CASE WHEN dr.ball_index BETWEEN we.ball_index + 1 AND we.ball_index + CASE WHEN
    we.format_name = 'ODI' THEN 24 ELSE 12 END THEN dr.runs_scored END) AS runs_after,
    COUNT(CASE WHEN dr.ball_index BETWEEN we.ball_index + 1 AND we.ball_index + CASE
    WHEN we.format_name = 'ODI' THEN 24 ELSE 12 END THEN 1 END) AS balls_after
    FROM wicket_events we JOIN delivery_runs dr ON we.innings_id = dr.innings_id GROUP
    BY 1, 2, we.ball_index
)
SELECT ww.format_name, t.team_name, ROUND(AVG((ww.runs_before /
NULLIF(ww.balls_before, 0)) * 6), 2) AS avg_rr_before_wicket,
ROUND(AVG((ww.runs_after / NULLIF(ww.balls_after, 0)) * 6), 2) AS avg_rr_after_wicket,
ROUND(AVG(((ww.runs_after / NULLIF(ww.balls_after, 0)) - (ww.runs_before /
NULLIF(ww.balls_before, 0))) * 6), 2) AS avg_rr_change,
COUNT(*) AS wicket_events
FROM wicket_windows ww JOIN Team t ON ww.team_id = t.team_id WHERE ww.balls_before > 0
AND ww.balls_after > 0 GROUP BY 1, 2
ORDER BY 1, avg_rr_change ASC;
```

Output:

format_name	team_name	avg_rr_before_wicket	avg_rr_after_wicket	avg_rr_change	wicket_events
ODI	Australia	6.55	5.57	-0.98	145
ODI	India	6.20	5.23	-0.97	136
ODI	Scotland	5.04	4.13	-0.90	42
ODI	Bangladesh	5.24	4.36	-0.88	148
ODI	United Arab Emirates	4.54	3.75	-0.79	48
ODI	Zimbabwe	5.56	4.81	-0.75	80
ODI	Ireland	5.58	4.84	-0.74	85
ODI	Sri Lanka	5.75	5.15	-0.60	143
ODI	West Indies	5.63	5.08	-0.55	123
ODI	Netherlands	4.88	4.36	-0.52	104
ODI	England	5.87	5.36	-0.50	151
ODI	South Africa	5.99	5.54	-0.45	149
ODI	New Zealand	5.86	5.52	-0.33	156
ODI	Pakistan	5.30	5.27	-0.03	146
ODI	Kenya	3.73	3.77	0.04	49
ODI	Canada	3.82	3.90	0.08	46
T20	Nepal	6.27	4.48	-1.80	32
T20	United Arab Emirates	6.27	4.65	-1.63	39
T20	United States of America	6.96	5.39	-1.57	33
T20	Australia	8.19	6.63	-1.55	93
T20	Hong Kong	5.73	4.74	-0.99	26
T20	Sri Lanka	7.12	6.19	-0.93	103
T20	Ireland	6.72	5.80	-0.91	84
T20	India	7.72	7.02	-0.70	94
T20	Zimbabwe	6.80	6.24	-0.57	74
T20	Scotland	7.03	6.46	-0.57	64
T20	England	7.98	7.46	-0.52	95
T20	West Indies	7.37	6.85	-0.52	96
T20	Bangladesh	6.60	6.13	-0.47	101
T20	South Africa	7.48	7.02	-0.46	91
T20	Netherlands	6.15	5.73	-0.42	88
T20	Papua New Guinea	5.21	4.84	-0.37	40
T20	New Zealand	7.68	7.34	-0.34	91
T20	Pakistan	7.29	7.01	-0.28	91
T20	Namibia	6.25	5.99	-0.26	60
T20	Oman	6.28	6.29	0.01	53
T20	Uganda	2.84	2.98	0.14	22
T20	Canada	6.38	7.44	1.06	17

38 rows in set (9.77 sec)

Short analysis:

Wicket losses cause sharper reductions in run rate in T20 matches due to limited recovery windows. ODI teams absorb wicket shocks more effectively, especially those with experienced batting line-ups. Rapid post-wicket stabilisation emerges as a key characteristic of successful teams.

3.4 Discussion of Query Design and Complexity

Across the suite, query design followed three consistent principles:

1. **Format-aware logic (ODI vs T20)**

Phase boundaries, window sizes, and thresholds were implemented using explicit format conditions to ensure correctness across formats.

2. **Event-level granularity with controlled aggregation**

All metrics ultimately trace back to Delivery-level records, then aggregate upwards (innings/match/team/player), preventing loss of detail and enabling reproducible computation.

3. Statistical reliability through filters

Many outputs apply minimum match/ball thresholds to reduce misleading rankings caused by small sample sizes.

Advanced SQL features were used where analytically necessary, including:

- **CTEs** to improve readability and isolate intermediate logic.
- **Window functions** for ranking, normalisation, and running totals.
- **Relational joins** that rely on resolved M:N structures (MatchTeam) and event decomposition (DeliveryExtra, Wicket, FieldingEvent).

3.5 Link to Application-Level Usage (Preparation for Stage 4)

Although Stage 4 presents the web application, Stage 3 establishes the analytical foundation it depends on. Each query in **queries.sql** is designed to return structured, display-ready outputs (ranked lists, grouped summaries, phase matrices) suitable for:

- tabular presentation,
- chart visualisation,
- and interactive filtering by format and phase.

This ensures the database is not only theoretically correct, but also practically usable as the analytics backend of the project.

3.6 Stage 3 Summary

Stage 3 implemented the Stage 2 relational design in MySQL and delivered a complete SQL analytics suite supporting all **15 research questions** from Stage 1. The resulting queries demonstrate:

- correct relational mapping (including M:N resolution via MatchTeam),
- robust event-level modelling (deliveries, wickets, extras, fielding),
- and advanced analytical capability through CTEs and window functions.

The next stage (Stage 4) builds directly on this foundation by presenting these query results through a user-facing web application.

Stage 4 – Web Application (Node.js Module)

4.1 Purpose and Scope

Stage 4 implements a working web application that queries the project database and presents World Cup cricket analytics through a clean, user-friendly interface. The purpose is to transform the SQL analysis (Stage 2/3) into an interactive system that supports the research motivation defined in Stage 1: extracting meaningful comparisons between ODI and T20 World Cup performance patterns using delivery-level evidence.

The application supports:

- A **Query Hub (Home Page)** listing the available research questions (Q1–Q15)
- A dedicated **Results Page per Question**
- Consistent display of:
 - query title + full research question
 - “Analyst’s Take” explanation (interpretation for non-technical users)
 - **full results table** (transparency and reproducibility)
 - **Chart.js visualisations** where relevant (pattern discovery)

Because the report has a strict length constraint, only **six representative query pages** are evidenced with screenshots (plus the Home page). These six were chosen to show variation in output structure (table-only vs chart+table) and variation in analytical focus (batting, bowling/phase analysis, match outcomes, etc.).

4.2 Technology Stack

Backend

- Node.js + Express.js (routing and server logic)
- MySQL (relational database)
- mysql2 (database driver / pool)

Frontend

- EJS (server-side templates for consistent rendering)
- Chart.js (interactive charts on selected questions)
- Custom CSS (dashboard styling, layout consistency, and responsiveness)
- Font Awesome (icons, navigation clarity)

Runtime Environment

- Coursera Lab with proxy routing (e.g., [/proxy/3000/](#)), requiring correct static asset setup so styling works both on the Home route and nested Results routes.

4.3 Application Architecture and Files

The application is structured so that database logic, query definitions, routing, and UI rendering are separated cleanly.

4.3.1 Backend Components

- **db.js**
 - Creates and exports a MySQL connection pool
 - Centralises database configuration and ensures reusable query execution
- **app_queries.js**
 - Stores SQL queries (Q1–Q15)
 - Provides a single source of truth for analytical logic used by the application
- **routes.js**
 - Handles routing for:
 - Query Hub (Home)
 - Query Results pages ([/query/:id](#))
 - Fetches query output from MySQL
 - Loads query metadata (title, full question, analyst explanation, chart config)
 - Sends both results + metadata into the EJS templates

4.3.2 Frontend Components

- **index.ejs**
 - Query Hub home page
 - Displays a readable menu of research questions with clickable navigation
- **results.ejs**
 - Standard layout used for every question page:
 1. Header + navigation back to Home
 2. Title + research question
 3. Analyst's Take
 4. Chart container (only if configured)
 5. Full data table (always rendered)
- **style.css**
 - Dashboard theme styling (layout, spacing, typography)
 - Responsive constraints to prevent charts/tables breaking on smaller screens
 - Visual consistency between Home and Results pages

4.4 Database Access and Privilege Considerations

The application is intentionally designed as a **read-only analytics interface**. It does not insert, update, or delete any records. The database account used by the web module is configured with only the minimum required privileges:

- SELECT privileges for the tables/views required by the analytical queries

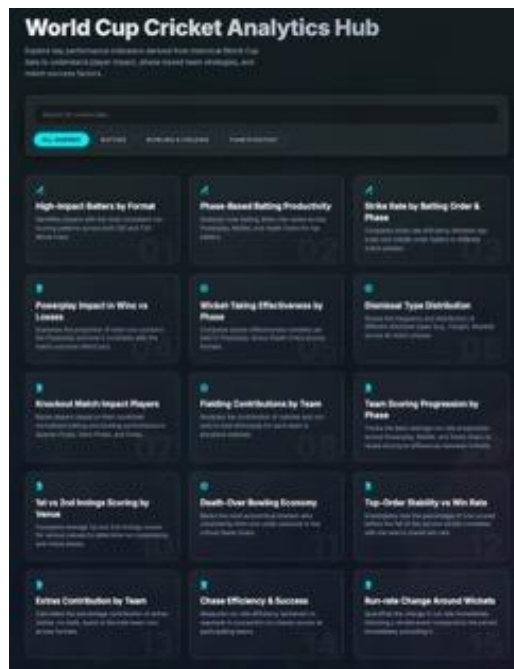
This supports good practice by reducing risk and ensuring analysis reproducibility.

4.5 User Flow and Application Behaviour

4.5.1 Query Hub (Home Page)

The Home page functions as a “research navigation hub”, listing each analytical question with a clear title. The goal is to reduce friction for users who are not technical: instead of running SQL manually, a user selects a question and immediately sees an interpreted results view.

Screenshot Evidence



- **Figure 4.1: Home Dashboard – World Cup Cricket Analytics Hub**
Shows the question list and navigation entry points.

4.5.2 Results Page (Per Question)

Each Results page follows a consistent structure to make the application feel like a cohesive analytics product rather than disconnected outputs. Every question page includes:

- Title and full research question (context)
- Analyst's Take (interpretation)
- Full results table (all rows returned by SQL)
- Visualisation section where configured (Chart.js)

This ensures:

- **Interpretability** (what does this mean?)
- **Transparency** (the raw outputs are visible)

4.6 Visual Analytics (Chart.js) Integration

Charts are used only when they add value beyond tables, especially for comparisons across formats or phases. Chart rendering is driven by centrally-defined metadata so that chart logic is reusable and does not require rewriting per query.

Examples of chart usage:

- Ranking comparisons (bar charts)
- Phase-based comparison visuals (grouped/stacked bars)
- Format-based contrast (ODI vs T20 toggles or grouped datasets)

Even when a chart shows only a subset for readability (e.g., top performers), the table still displays **all rows**, preserving completeness.

4.7 Screenshots Included (Home + 6 Questions)

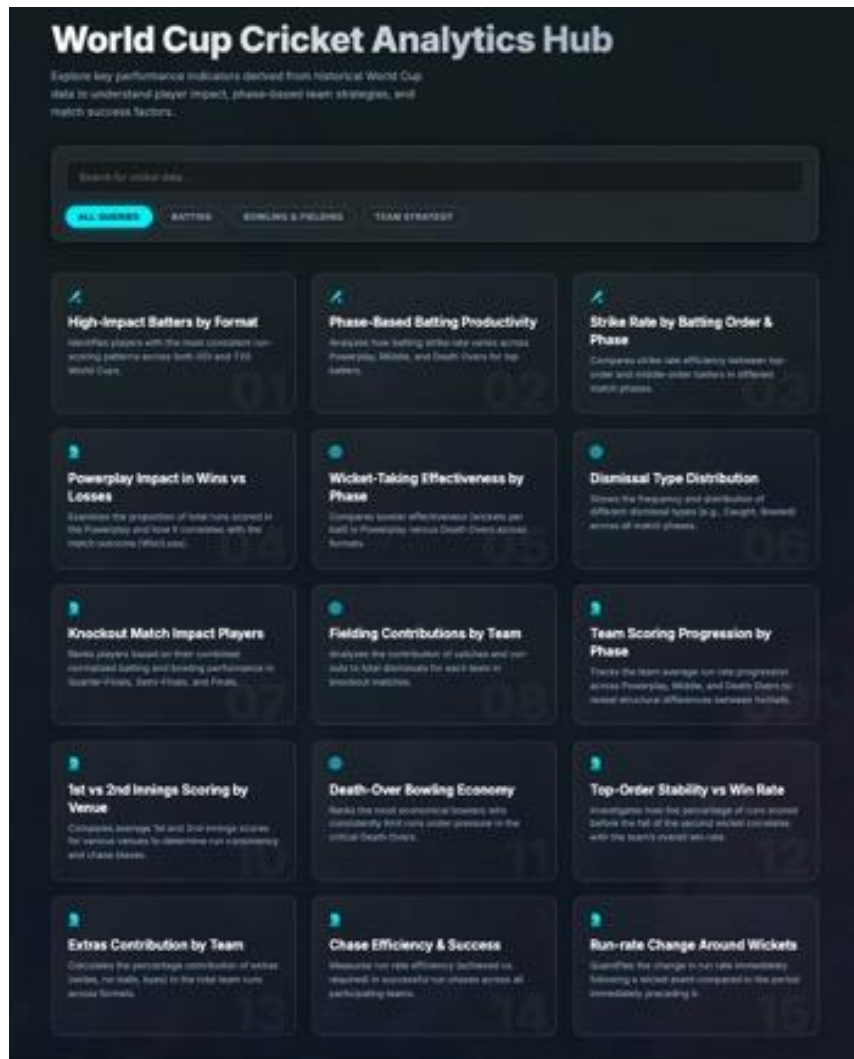
The report includes screenshots for:

- 1 Home page
- 6 Results pages (representative sample)

Only six were included due to report length limitations, but the interface supports all query pages using the same Results template and routing structure.

Included Evidence Set

- **Figure 4.1: Home Dashboard – World Cup Cricket Analytics Hub**



(Overview of all 15 research questions with category filters and search functionality)

- **Figure 4.2: Results Page – Q1 High-Impact Batters by Format**



(Bar Chart of Weighted Impact Score with Supporting Results Table)

- **Figure 4.3: Results Page – Q2 Phase-Based Batting Productivity**



(Scatter Plot of Runs vs Strike Rate by Match Phase with Supporting Results Table)

- ### Q6. Dismissal Type Distribution

What dismissal types dominate in 50% and T20 World Cups, and how does their distribution vary across match phases?

Dr. ANALYST'S TAKE

Caught dismissals dominate across all phases, reflecting sustained fielding pressure as the primary wicket mechanism. T20 death overs show a marked increase in run-outs, highlighting forced errors during acceleration. ODIs maintain a more stable dismissal mix due to longer innings durations and reduced urgency.

DISMISSAL TYPE DISTRIBUTION BY PHASE

[50% World Cup] [T20 World Cup]

Phase	Format	Dismissal Type	Count	Percentage (%)
Powerplay	50%	Caught	250	62.47
		Run out	75	18.75
		Bowled	81	20.25
		LBW	18	4.54
		Caught and bowled	14	3.54
		Stumped	8	2.01
		Hit wicket	0	0.00
Middle Overs	50%	Caught	220	55.25
		Run out	220	55.25
		Bowled	150	37.50
		LBW	80	20.00
Death Overs	50%	Caught	180	45.00
		Run out	120	30.00
		Bowled	100	25.00
		LBW	60	15.00
Powerplay	T20	Caught	180	45.00
		Run out	120	30.00
		Bowled	100	25.00
		LBW	60	15.00
		Caught and bowled	40	10.00
		Stumped	20	5.00
		Hit wicket	10	2.50
Middle Overs	T20	Caught	150	37.50
		Run out	100	25.00
		Bowled	80	20.00
		LBW	50	12.50
Death Overs	T20	Caught	100	25.00
		Run out	150	37.50
		Bowled	120	30.00
		LBW	80	20.00

(Stacked Bar Chart of Dismissal Types by Match Phase with Supporting Results Table)

- **Figure 4.5: Results Page – Q7 Knockout Match Impact Players**



(Lollipop Chart of Combined Batting and Bowling Impact Scores with Supporting Results Table)

- *Figure 4.6: Results Page – Q9 Team Scoring Progression by Phase*



(Line Chart of Run Rate Progression Across Match Phases with Supporting Results Table)

- **Figure 4.7: Results Page – Q15 Momentum Impact After Wickets**



(Dumbbell Chart Comparing Run Rate Before and After Wickets with Supporting Results Table)

4.8 Robustness and Error Handling

The application includes defensive behaviour to remain stable under typical issues:

- Invalid query IDs return a controlled Not Found response
- Database errors are handled without crashing the server
- Empty result sets still render the page layout with an appropriate message
- Static assets load correctly on nested routes so results pages remain styled consistently

This matters because assessment demonstrations often involve rapid navigation across multiple pages and queries.

4.9 Evaluation Against Stage 4 Requirements

Stage 4 requires a Node.js web module that queries the database and presents results addressing the project motivation. This implementation meets the requirement by:

- Running SQL queries dynamically through Express routes
- Presenting results in an interpretable analytics layout
- Supporting ODI vs T20 comparative exploration where applicable
- Using Chart.js visualisations to communicate patterns efficiently
- Showing reproducible raw outputs through full results tables

4.10 Summary

Stage 4 delivers a complete analytics web application that bridges the gap between SQL-based research and user-facing insight. The system provides a structured Query Hub, consistent Results pages, narrative interpretation, and interactive charts where needed. Evidence is provided through screenshots of the Home page and six representative questions, selected to demonstrate both visual and table-based outputs while staying within report length constraints.
